Fondamenti di Informatica

Appunti del corso · a.a. 2026/27 · Mobius

Appunti personali e non ufficiali, rivisti e confrontati con il programma ufficiale del corso. Non sostituiscono le lezioni né i testi consigliati.

Indice

1. Insiemi, relazioni e funzioni
2. Strutture algebriche e calcolo combinatorio
3. Calcolatori, algoritmi e primi programmi in C++
4. Funzioni, array, stringhe, file e strutture
5. Puntatori, memoria dinamica, ricorsione e template
6. Classi e oggetti
7. Tipi di dato astratti, liste, pile e code

1 · Insiemi, relazioni e funzioni

In questo modulo. Non servono prerequisiti oltre all'aritmetica delle superiori. È la prima parte di matematica discreta del corso: il linguaggio degli insiemi, che poi torna nella programmazione (un tipo di dato è un insieme di valori, una funzione C++ realizza una funzione matematica, una relazione d'ordine è quella che usa un algoritmo di ordinamento). Si vedono gli insiemi e le loro operazioni, il prodotto cartesiano, le relazioni con le loro proprietà, le relazioni di equivalenza con le classi e le partizioni, le relazioni d'ordine con massimi, minimi, maggioranti ed estremi, e le funzioni con immagine, controimmagine, iniettività e suriettività. Alla fine del modulo si sanno risolvere gli esercizi "sì o no" della prova di teoria su un insieme, un ordine o una funzione dati, e contare relazioni e funzioni fra insiemi finiti.

> **Aggiunta in revisione · fonte: slide del corso (Elementi di matematica discreta, 1.1 Teoria degli insiemi, 1.2 Relazioni e funzioni), foglio `esercizi_matematica`; C. Delizia e altri, *Matematica discreta*, McGraw-Hill, capp. 1-2 (FI-01, FI-02)**
>
> Non ci sono appunti delle lezioni di questo corso: il modulo è scritto in revisione dal materiale del corso, nello stile degli altri. Gli esercizi sono nuovi, sul modello delle prove 2016-2020; le soluzioni sono controllate con `strumenti/verifica.py`.

1.1 Insiemi e sottoinsiemi

Un **insieme** è una collezione di oggetti, i suoi **elementi**. Si descrive per elencazione, $S = \{g, 1, +\}$, oppure con una proprietà caratteristica, $T = \{x : x \text{ è una vocale}\}$. L'ordine in cui si elencano gli elementi non conta, e un elemento ripetuto conta una volta sola: $\{1, 2\} = \{2, 1\} = \{1, 1, 2\}$. Si scrive $x \in A$ se x appartiene ad A, $x \notin A$ altrimenti. L'**insieme vuoto** $\emptyset$ non ha elementi; un insieme con un solo elemento, come $\{m\}$, si dice **singoletto**.

Gli insiemi numerici del corso: $\mathbb{N}_0 = \{0, 1, 2, \dots\}$ i naturali con lo zero, $\mathbb{N} = \{1, 2, 3, \dots\}$ i naturali senza lo zero, $\mathbb{Z}$ gli interi, $\mathbb{Q}$ i razionali, $\mathbb{R}$ i reali, $\mathbb{C}$ i complessi. Attenzione: in altri corsi $\mathbb{N}$ comprende lo zero; qui no.

La **cardinalità** (o ordine) $|A|$ è il numero degli elementi di A: $|\{g, 1, +\}| = 3$, $|\emptyset| = 0$. Un insieme è finito se la sua cardinalità è un numero naturale, altrimenti è infinito.

A è **contenuto** in B (è un **sottoinsieme** di B), $A \subseteq B$, se ogni elemento di A è elemento di B:

$$A \subseteq B \iff (\forall x)(x \in A \Rightarrow x \in B).$$

A è contenuto **strettamente** in B, $A \subset B$, se $A \subseteq B$ e $A \neq B$, cioè se in più c'è almeno un elemento di B che non sta in A. Per ogni insieme A valgono $\emptyset \subseteq A$ e $A \subseteq A$, e l'inclusione è transitiva: se $A \subseteq B$ e $B \subseteq C$ allora $A \subseteq C$. Due insiemi sono **uguali** se hanno gli stessi elementi, cioè se $A \subseteq B$ e $B \subseteq A$: è il metodo della **doppia inclusione**, che si usa per dimostrare le uguaglianze fra insiemi.

Bisogna distinguere appartenenza e inclusione. Se $x \in A$ allora $\{x\} \subseteq A$, ma non $\{x\} \in A$ (a meno che l'insieme $\{x\}$ sia proprio uno degli elementi di A). E $\emptyset \neq \{\emptyset\}$: il primo non ha elementi, il secondo ne ha uno, l'insieme vuoto.

L'**insieme delle parti** (o insieme potenza) di A è l'insieme di tutti i sottoinsiemi di A:

$$\mathcal{P}(A) = \{X : X \subseteq A\}.$$

Per $H = \{3, 5, 8\}$: $\mathcal{P}(H) = \{\emptyset, \{3\}, \{5\}, \{8\}, \{3, 5\}, \{3, 8\}, \{5, 8\}, H\}$. Contiene sempre $\emptyset$ e A, quindi non è mai vuoto.

> **Cardinalità dell'insieme delle parti**
>
> Se $|A| = n$, allora $|\mathcal{P}(A)| = 2^n$.

Per costruire un sottoinsieme si decide, per ciascuno degli n elementi, se metterlo dentro o no: sono n scelte indipendenti fra due possibilità, quindi $2 \cdot 2 \cdots 2 = 2^n$ sottoinsiemi (è il principio del prodotto del modulo 2).

1.2 Operazioni fra insiemi

Dati due insiemi A e B:

- **unione** $A \cup B = \{x : x \in A \vee x \in B\}$;
- **intersezione** $A \cap B = \{x : x \in A \wedge x \in B\}$; se è vuota, A e B sono **disgiunti**;
- **differenza** $A \setminus B = \{x : x \in A \wedge x \notin B\}$;
- **complemento** di A rispetto a un insieme universo U che lo contiene: $\overline{A} = U \setminus A$;
- **differenza simmetrica** $A \Delta B = (A \setminus B) \cup (B \setminus A)$, gli elementi che stanno in uno solo dei due.

Le operazioni si visualizzano con i **diagrammi di Venn**. Valgono le proprietà commutativa e associativa di unione e intersezione, le distributive

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C), \qquad A \cup (B \cap C) = (A \cup B) \cap (A \cup C),$$

e le **leggi di De Morgan**

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \qquad \overline{A \cap B} = \overline{A} \cup \overline{B}.$$

Sono le stesse leggi della logica (unione come “o”, intersezione come “e”, complemento come “non”), e torneranno nelle condizioni dei programmi: `!(a || b)` equivale a `!a && !b`.

Per contare gli elementi di un’unione di insiemi finiti si usa il **principio di inclusione-esclusione**:

$$|A \cup B| = |A| + |B| - |A \cap B|, \qquad |A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|.$$

Sommando $|A|$ e $|B|$ gli elementi comuni si contano due volte, e vanno tolti una volta.

Prodotto cartesiano. Una **coppia ordinata** (a, b) è diversa da (b, a) se $a \neq b$: conta l’ordine, a differenza dell’insieme $\{a, b\}$. Il prodotto cartesiano di A e B è l’insieme delle coppie con il primo elemento in A e il secondo in B:

$$A \times B = \{(a, b) : a \in A, b \in B\}, \qquad |A \times B| = |A| \cdot |B|.$$

In generale $A \times B \neq B \times A$. Il prodotto $A \times A$ si scrive anche A^2, e allo stesso modo si definiscono le terne, le n -uple e A^n.

Partizioni. Una **partizione** di A è una famiglia di sottoinsiemi di A non vuoti, disgiunti a due a due, la cui unione è A. Per esempio $\{\{1, 4\}, \{2\}, \{3, 5\}\}$ è una partizione di $\{1, 2, 3, 4, 5\}$; $\{\{1, 4\}, \{2, 4\}, \{3, 5\}\}$ no, perché il 4 sta in due blocchi.

1.3 Relazioni

Una **relazione** fra A e B è un sottoinsieme $R \subseteq A \times B$; si scrive $a \, R \, b$ per $(a, b) \in R$. Se $A = B$ si parla di relazione su A. Poiché le relazioni sono i sottoinsiemi di $A \times B$, il loro numero è

$$|\mathcal{P}(A \times B)| = 2^{|A| \cdot |B|}.$$

La relazione **inversa** è $R^{-1} = \{(b, a) : (a, b) \in R\}$.

Una relazione su un insieme finito si può rappresentare con l’elenco delle coppie, con una **matrice booleana** (riga a, colonna b, 1 se $a \, R \, b$) o con un **grafo orientato** (una freccia da a a b se $a \, R \, b$, un cappio se $a \, R \, a$).

Una relazione R su A è:

Proprietà	Definizione	Nella matrice o nel grafo
riflessiva	$a \, R \, a$ per ogni $a \in A$	diagonale tutta a 1; un cappio su ogni nodo

Proprietà	Definizione	Nella matrice o nel grafo
simmetrica	$a\,R\,b \Rightarrow b\,R\,a$	matrice simmetrica; frecce sempre in coppia
antisimmetrica	$a\,R\,b \wedge b\,R\,a \Rightarrow a = b$	mai due frecce opposte fra nodi diversi
transitiva	$a\,R\,b \wedge b\,R\,c \Rightarrow a\,R\,c$	ogni percorso di due frecce ha la "scorciatoia"

Simmetrica e antisimmetrica non sono l'una il contrario dell'altra: la relazione di uguaglianza è entrambe le cose, e una relazione può non essere né l'una né l'altra.

Relazioni di equivalenza. Una relazione riflessiva, simmetrica e transitiva si dice **di equivalenza** (si scrive spesso $a \sim b$). La **classe di equivalenza** di a è l'insieme degli elementi equivalenti ad a, $[a] = \{x \in A : x \sim a\}$; l'insieme delle classi è l'**insieme quoziente** $A/\!\sim$.

> **Equivalenze e partizioni**
>
> Le classi di una relazione di equivalenza su A formano una partizione di A. Viceversa, ogni partizione di A definisce un'equivalenza: due elementi sono equivalenti se stanno nello stesso blocco.

Le classi non sono vuote ($a \in [a]$ per la riflessività), coprono A, e due classi sono disgiunte o coincidono: se $c \in [a] \cap [b]$, per simmetria e transitività ogni elemento equivalente ad a lo è anche a b, quindi $[a] = [b]$.

Esempio importante: su $\mathbb{Z}$, $a \equiv b \pmod n$ se n divide $a - b$ (**congruenza modulo** n). È un'equivalenza, le classi sono n, una per ogni resto della divisione per n: $[0], [1], \dots, [n-1]$. Per $n = 2$ le classi sono i pari e i dispari.

1.4 Relazioni d'ordine

Una relazione riflessiva, **antisimmetrica** e transitiva è una **relazione d'ordine** (si scrive spesso $\leq$), e $(A, \leq)$ è un **insieme ordinato**. Due elementi sono **confrontabili** se $a \leq b$ oppure $b \leq a$. L'ordine è **totale** se tutte le coppie sono confrontabili, **parziale** altrimenti.

Esempi: $(\mathbb{N}, \leq)$ con l'ordine usuale è totale. $(\mathbb{N}, \mid)$ con la divisibilità ($a \mid b$ se a divide b) è parziale: 2 e 3 non sono confrontabili. $(\mathcal{P}(A), \subseteq)$ è parziale se $|A| \geq 2$: $\{1\}$ e $\{2\}$ non sono confrontabili.

Un ordine su un insieme finito si disegna con il **diagramma di Hasse**: un punto per elemento, b più in alto di a se $a < b$, e un segmento solo fra a e b quando b "copre" a, cioè $a < b$ senza elementi in mezzo. Riflessività e transitività restano sottintese. Per i divisori di 12 con la divisibilità, $\{1, 2, 3, 4, 6, 12\}$: 1 in basso, sopra 2 e 3, sopra il 2 il 4 e il 6 (il 6 anche sopra il 3), in cima 12, collegato a 4 e 6.

Elementi particolari. Sia $(A, \leq)$ ordinato e $X \subseteq A$ non vuoto.

Nome	Definizione
massimo di X	$M \in X$ e $x \leq M$ per ogni $x \in X$
minimo di X	$m \in X$ e $m \leq x$ per ogni $x \in X$
massimale di X	elemento di X senza elementi di X strettamente maggiori
minimale di X	elemento di X senza elementi di X strettamente minori
maggiorante di X in A	$u \in A$ confrontabile con ogni $x \in X$ e $x \leq u$ per ogni x
minorante di X in A	$v \in A$ confrontabile con ogni $x \in X$ e $v \leq x$ per ogni x
estremo superiore	minimo dei maggioranti (se esiste)
estremo inferiore	massimo dei minoranti (se esiste)

Massimo e minimo, se esistono, sono unici e appartengono a X; i massimali e i minimali possono essere più di uno. Maggioranti e minoranti stanno in A, non necessariamente in X. Se il massimo esiste è anche l'unico massimale e l'estremo superiore; in un ordine totale massimale e massimo coincidono.

Nei divisori di 12, per $X = \{4, 6\}$: non c'è massimo (4 e 6 non sono confrontabili), entrambi sono massimali; i maggioranti sono i multipli comuni in A, cioè solo 12, che è l'estremo superiore; i minoranti sono 1 e 2, e l'estremo inferiore è 2. Con la divisibilità, estremo superiore ed estremo inferiore sono il minimo comune multiplo e il massimo comune divisore.

1.5 Funzioni

Una **funzione** (o applicazione) $f : A \to B$ è una relazione fra A e B in cui ogni elemento di A è in relazione con **uno e un solo** elemento di B, la sua **immagine** $f(a)$. A è il **dominio**, B il **codominio**. Non tutte le relazioni sono funzioni: in una funzione da ogni elemento del dominio parte esattamente una freccia.

L'**immagine** di un sottoinsieme $X \subseteq A$ è $f(X) = \{f(x) : x \in X\}$, e $f(A)$ si chiama immagine di f. La **controimmagine** di $Y \subseteq B$ è $f^{-1}(Y) = \{a \in A : f(a) \in Y\}$: è sempre definita, anche quando la funzione non è invertibile, e può essere vuota.

Proprietà	Definizione	Significato
iniettiva	$a_1 \neq a_2 \Rightarrow f(a_1) \neq f(a_2)$	elementi diversi hanno immagini diverse
suriettiva	$f(A) = B$	ogni elemento del codominio è immagine di qualcuno
biiettiva	iniettiva e suriettiva	corrispondenza uno a uno

Per dimostrare l'iniettività conviene la forma equivalente $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$; per smentirla basta trovare due elementi diversi con la stessa immagine. Per esempio $f : \mathbb{Z} \to \mathbb{Z}$, $f(x) = 2x + 1$ è iniettiva ($2a_1 + 1 = 2a_2 + 1 \Rightarrow a_1 = a_2$) ma non suriettiva (i pari non sono immagini); $g : \mathbb{Z} \to \mathbb{N}_0$, $g(x) = |x|$ è suriettiva ma non iniettiva ($g(-1) = g(1)$).

La **composizione** di $f : A \to B$ e $g : B \to C$ è $g \circ f : A \to C$, $(g \circ f)(a) = g(f(a))$; non è commutativa. Una funzione biiettiva ha la **funzione inversa** $f^{-1} : B \to A$, con $f^{-1}(f(a)) = a$; solo le biiezioni sono invertibili.

Contare le funzioni. Se $|A| = k$ e $|B| = n$:

- le funzioni $A \to B$ sono n^k (ogni elemento di A sceglie la sua immagine fra n);
- quelle iniettive sono $n(n-1)\cdots(n-k+1)$ se $k \leq n$, e nessuna se $k > n$;
- se $k = n$ le biiezioni sono $n!$.

Il caso $k > n$ è il **principio dei cassetti**: se si mettono k oggetti in $n < k$ cassetti, almeno un cassetto ne contiene due.

Fra insiemi infiniti si confrontano le cardinalità con le biiezioni: due insiemi sono **equipotenti** se esiste una biiezione fra loro. $\mathbb{N}$, $\mathbb{Z}$ e $\mathbb{Q}$ sono equipotenti (insiemi **numerabili**), $\mathbb{R}$ no: è "più grande".

Esempi ed esercizi svolti

> **Aggiunta in revisione · fonte: esercizi nuovi sul modello delle prove intermedie 2016-2019 e del foglio `esercizi_matematica`; risposte controllate con `strumenti/verifica.py` (FI-01, FI-02)**

Esercizio 1. Sì o no sull'insieme delle parti. $A = \{2, x, \#\}$, $\mathcal{P}(A)$ con $X \, R \, Y \iff X \subseteq Y$.

Affermazione	Risposta	Perché
$(\mathcal{P}(A), R)$ è un insieme ordinato	sì	l'inclusione è riflessiva, antisimmetrica, transitiva
è totalmente ordinato	no	$\{2\}$ e $\{x\}$ non sono confrontabili
A è il massimo	sì	ogni sottoinsieme è contenuto in A
$\{2\}$ è l'unico minimale	no	l'unico minimale è $\emptyset$, che è anche il minimo
$(\{2, x\}, \{x, \#\}) \in R$	no	$2 \notin \{x, \#\}$

Affermazione	Risposta	Perché
$\lvert\mathcal{P}(A)\rvert = 8$	sì	2^3
i maggioranti di $\{\{2\},\{x\}\}$ sono $\{2,x\}$ e A	sì	sono i sottoinsiemi che contengono 2 e x
l'estremo superiore di $\{\{2\},\{x\}\}$ è $\{2,x\}$	sì	il minimo dei maggioranti; in generale è l'unione

Con $f : \mathcal{P}(A) \to \mathbb{N}_0$, $f(X) = \lvert X\rvert$:

Affermazione	Risposta	Perché
$f(\emptyset) = 1$	no	$f(\emptyset) = 0$
$f(\{\{x\},\{2,\#\}\}) = \{1,2\}$	sì	le cardinalità dei due elementi
$f^{-1}(\{2\}) = \{\{2,x\},\{2,\#\},\{x,\#\}\}$	sì	i sottoinsiemi con due elementi
$f^{-1}(\{5\}) = \emptyset$	sì	nessun sottoinsieme ha 5 elementi
f è iniettiva	no	$f(\{2\}) = f(\{x\}) = 1$
f è suriettiva	no	4 non è immagine di nulla

Esercizio 2. Riconoscere un'equivalenza. Su $A = \{1,2,3,4\}$, $R = \{(1,1),(2,2),(3,3),(4,4),(1,2),(2,1),(3,4),(4,3)\}$.

Riflessiva: le quattro coppie (a,a) ci sono. Simmetrica: $(1,2)$ e $(2,1)$, $(3,4)$ e $(4,3)$. Transitiva: le uniche catene sono del tipo $1\,R\,2\,R\,1$, che richiede $1\,R\,1$, presente, e così via. È un'equivalenza, con classi $[1] = \{1,2\}$ e $[3] = \{3,4\}$: l'insieme quoziente è $\{\{1,2\},\{3,4\}\}$, una partizione di A. Non è antisimmetrica: $(1,2)$ e $(2,1)$ con $1 \neq 2$.

Esercizio 3. Proprietà di tre relazioni su $\mathbb{Z}$.

a. $a\,R\,b \iff a - b$ è pari: riflessiva (0 è pari), simmetrica ($b - a = -(a-b)$), transitiva ($a - c = (a-b) + (b-c)$): equivalenza, con classi pari e dispari. È la congruenza modulo 2.

b. $a\,S\,b \iff a < b$: non riflessiva, non simmetrica, antisimmetrica (la premessa $a < b \wedge b < a$ non è mai vera), transitiva. Non è un ordine perché non è riflessiva: è un ordine **stretto**.

c. $a\,T\,b \iff \lvert a - b\rvert \leq 1$: riflessiva e simmetrica, ma non transitiva: $0\,T\,1$ e $1\,T\,2$, ma non $0\,T\,2$.

Esercizio 4. Divisori con la divisibilità. In $D = \{1,2,3,4,6,12\}$ ordinato da $\mid$, trovare massimo, minimo e, per $X = \{2,3\}$ e $Y = \{2,4\}$, maggioranti, minoranti ed estremi.

D ha massimo 12 e minimo 1. $X = \{2,3\}$: maggioranti 6 e 12, estremo superiore 6; minoranti solo 1, estremo inferiore 1; X non ha massimo né minimo. $Y = \{2,4\}$: $2 \mid 4$, quindi Y ha minimo 2 e massimo 4; i maggioranti sono 4 e 12 (estremo superiore 4, che appartiene a Y), i minoranti 1 e 2 (estremo inferiore 2).

Esercizio 5. Contare relazioni e funzioni. $\lvert A\rvert = 3$, $\lvert B\rvert = 2$.

Relazioni fra A e B: $2^{3\cdot 2} = 64$. Funzioni $A \to B$: $2^3 = 8$; nessuna è iniettiva (principio dei cassetti); le suriettive sono le 8 meno le 2 costanti, cioè 6. Funzioni $B \to A$: $3^2 = 9$, di cui iniettive $3 \cdot 2 = 6$, e nessuna suriettiva.

Esercizio 6. Inclusione-esclusione. Su 40 studenti, 25 hanno usato il C++, 18 il Python e 10 entrambi. Quanti nessuno dei due?

$\lvert C \cup P\rvert = 25 + 18 - 10 = 33$, quindi $40 - 33 = 7$ studenti non hanno usato nessuno dei due. Solo il C++: $25 - 10 = 15$.

Esercizio 7. Una dimostrazione per doppia inclusione. Dimostrare che $A \setminus (B \cup C) = (A \setminus B) \cap (A \setminus C)$.

($\subseteq$) Se $x \in A \setminus (B \cup C)$, allora $x \in A$ e $x \notin B \cup C$, cioè $x \notin B$ e $x \notin C$; quindi $x \in A \setminus B$ e $x \in A \setminus C$. ($\supseteq$) Se x sta in entrambe le differenze, allora $x \in A$, $x \notin B$, $x \notin C$, quindi $x \notin B \cup C$ e $x \in A \setminus (B \cup C)$. È la legge di De Morgan dentro A.

Esercizio 8. Composizione. $f : \mathbb{Z} \to \mathbb{Z}$, $f(x) = 2x + 1$ e $g : \mathbb{Z} \to \mathbb{N}_0$, $g(x) = |x|$. Calcolare $g \circ f$ e dire se è iniettiva.

$(g \circ f)(x) = |2x + 1|$. Non è iniettiva: $(g \circ f)(0) = 1 = (g \circ f)(-1)$. La composizione di una funzione iniettiva con una non iniettiva può perdere l'iniettività; la composizione di due funzioni iniettive è sempre iniettiva.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; risposte controllate con `strumenti/verifica.py` (FI-01, FI-02)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. È un ordine? Disegnare il diagramma. Su $A = \{a, b, c, d\}$: $R = \{(a,a), (b,b), (c,c), (d,d), (a,b), (a,c), (a,d), (b,d), (c,d)\}$.

Passo 1: riflessiva. Ci sono tutte le coppie (x, x): sì.

Passo 2: antisimmetrica. Fra elementi diversi c'è sempre al più una delle due coppie: sì.

Passo 3: transitiva. Le catene di due coppie con elementi diversi sono $a\,R\,b\,R\,d$ e $a\,R\,c\,R\,d$; entrambe richiedono (a, d), che c'è: sì. È un ordine.

Passo 4: Hasse. a in basso, sopra b e c (non confrontabili fra loro), in cima d. Il segmento a-d non si disegna: si ottiene passando da b o da c.

Passo 5: elementi particolari. Minimo a, massimo d. Per $X = \{b, c\}$: due massimali e due minimali (b e c), estremo superiore d, estremo inferiore a. L'ordine non è totale.

Guidato 2. Le classi della congruenza modulo 4. Su $A = \{0, 1, \dots, 11\}$, $a \sim b$ se $4 \mid (a - b)$.

Passo 1: è un'equivalenza. È la congruenza modulo 4 ristretta ad A.

Passo 2: le classi. Si raggruppano gli elementi per resto: $[0] = \{0, 4, 8\}$, $[1] = \{1, 5, 9\}$, $[2] = \{2, 6, 10\}$, $[3] = \{3, 7, 11\}$.

Passo 3: il controllo. Sono 4 classi di 3 elementi, disgiunte, con unione A: $4 \times 3 = 12 = |A|$.

Guidato 3. Immagine e controimmagine. $f : \{1, 2, 3, 4, 5, 6\} \to \mathbb{N}_0$, $f(n) = n \bmod 3$.

Passo 1: la tabella. $f(1) = 1$, $f(2) = 2$, $f(3) = 0$, $f(4) = 1$, $f(5) = 2$, $f(6) = 0$.

Passo 2: immagini. $f(\{1, 4, 6\}) = \{0, 1\}$; l'immagine di f è $\{0, 1, 2\}$.

Passo 3: controimmagini. $f^{-1}(\{0\}) = \{3, 6\}$; $f^{-1}(\{1, 2\}) = \{1, 2, 4, 5\}$; $f^{-1}(\{7\}) = \emptyset$.

Passo 4: proprietà. Non è iniettiva ($f(1) = f(4)$), non è suriettiva su $\mathbb{N}_0$. Le controimmagini dei singoli valori, $\{3, 6\}$, $\{1, 4\}$, $\{2, 5\}$, formano una partizione del dominio: è sempre così, e la relazione "hanno la stessa immagine" è un'equivalenza.

Riepilogo

Un insieme è determinato dai suoi elementi; l'inclusione si controlla elemento per elemento e l'uguaglianza con la doppia inclusione; l'insieme delle parti di un insieme con n elementi ne ha 2^n. Unione, intersezione, differenza, complemento e differenza simmetrica seguono le leggi della logica, comprese le leggi di De Morgan; il principio di inclusione-esclusione conta gli elementi di un'unione. Il prodotto cartesiano è fatto di coppie ordinate e ha $|A| \cdot |B|$ elementi. Una relazione è un sottoinsieme del prodotto cartesiano: fra A e B ce ne sono $2^{|A||B|}$. Le relazioni riflessive, simmetriche e transitive sono equivalenze, e le loro classi formano una partizione; le riflessive, antisimmetriche e transitive sono ordini, totali o parziali, con massimi, minimi, massimali, minimali, maggioranti, minoranti ed estremi. Una funzione associa a ogni elemento del dominio una sola immagine; può essere iniettiva, suriettiva o biiettiva, e solo nell'ultimo caso è invertibile; fra insiemi finiti le funzioni sono n^k.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (FI-01, FI-02)**

Conteggi

$$|\mathcal{P}(A)| = 2^{|A|}, \qquad |A \times B| = |A||B|, \qquad \#\text{relazioni} = 2^{|A||B|}, \qquad \#\text{funzioni } A \to B = |B|^{|A|},$$

$$\#\text{iniettive} = \frac{n!}{(n-k)!}\ (k \le n), \qquad \#\text{biiezioni} = n!, \qquad |A \cup B| = |A| + |B| - |A \cap B|.$$

Leggi

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \quad \overline{A \cap B} = \overline{A} \cup \overline{B}, \quad A \cap (B \cup C) = (A \cap B) \cup (A \cap C), \quad A \,\Delta\, B = (A \setminus B) \cup (B \setminus A).$$

Proprietà delle relazioni

Tipo	Riflessiva	Simmetrica	Antisimmetrica	Transitiva
equivalenza	sì	sì	–	sì
ordine	sì	–	sì	sì
ordine stretto	no	–	sì	sì

Termini per l’orale

Termine	In una riga
Sottoinsieme proprio	contenuto e diverso
Partizione	blocchi non vuoti, disgiunti, che coprono l’insieme
Classe di equivalenza, insieme quoziente	elementi equivalenti a uno dato; insieme delle classi
Confrontabili	$a \le b$ oppure $b \le a$
Massimale	nessuno strettamente più grande in X
Maggiorante	in A, più grande o uguale di tutti gli elementi di X
Estremo superiore	il più piccolo dei maggioranti
Controimmagine	elementi del dominio con immagine in Y
Principio dei cassetti	$k > n$ oggetti in n cassetti: un cassetto ne ha due
Numerabile	in biiezione con $\mathbb{N}$

Domande che tornano spesso

- Differenza fra $\in$ e $\subseteq$; fra $\emptyset$ e $\{\emptyset\}$.
- Quanti sono i sottoinsiemi, le relazioni, le funzioni fra insiemi finiti?
- Definire equivalenza e ordine; legame fra equivalenze e partizioni.
- Massimo e massimale, maggiorante ed estremo superiore: differenze, con un esempio.
- Iniettiva, suriettiva, biiettiva; quando esiste l’inversa?

2 · Strutture algebriche e calcolo combinatorio

In questo modulo. Si usano gli insiemi, il prodotto cartesiano e le funzioni del modulo 1. La prima parte studia le operazioni su un insieme e le strutture che ne nascono: semigruppi, monoidi, gruppi, anelli, corpi e campi. Sono le "regole del gioco" dei tipi di dato: gli interi del C++ con la somma, le stringhe con la concatenazione, i resti modulo n dell'aritmetica dei calcolatori. La seconda parte è il calcolo combinatorio, cioè l'arte di contare senza elencare: principi della somma e del prodotto, disposizioni, permutazioni e combinazioni, semplici e con ripetizione, coefficienti binomiali. Alla fine del modulo si sa riconoscere la struttura di un insieme con le sue operazioni e scegliere la formula giusta per un problema di conteggio, come nell'esercizio di combinatoria delle prove scritte.

> **Aggiunta in revisione · fonte: slide del corso (1.3 Introduzione alle strutture algebriche, 1.4 Elementi di calcolo combinatorio), foglio `esercizi_matematica`; C. Delizia e altri, *Matematica discreta*, McGraw-Hill (FI-03, FI-04)**
>
> Il modulo è scritto in revisione dal materiale del corso; gli esercizi sono nuovi, sul modello delle prove 2016-2020, e controllati con `strumenti/verifica.py`.

2.1 Operazioni

Un'**operazione (binaria) interna** su un insieme A è una funzione $* : A \times A \to A$: a ogni coppia di elementi di A associa un elemento di A, che si scrive $a * b$. "Interna" vuol dire che il risultato resta in A: la sottrazione è un'operazione interna su $\mathbb{Z}$ ma non su $\mathbb{N}_0$, perché $2 - 5 \notin \mathbb{N}_0$. Un'operazione **esterna** combina elementi di un altro insieme con quelli di A, $K \times A \to A$: il prodotto di un numero reale per un vettore è l'esempio tipico. Su un insieme finito un'operazione si descrive con la sua **tabella** (di Cayley): riga a, colonna b, nella casella $a * b$.

Le proprietà che contano:

Proprietà	Definizione
associativa	$(a * b) * c = a * (b * c)$ per ogni a, b, c
commutativa	$a * b = b * a$ per ogni a, b
elemento neutro	esiste $e \in A$ con $a * e = e * a = a$ per ogni a
elemento simmetrico di a	$a' \in A$ con $a * a' = a' * a = e$

L'elemento neutro, se c'è, è unico: se e ed e' sono neutri, $e = e * e' = e'$. Se l'operazione è associativa anche il simmetrico di un elemento è unico. Il simmetrico si chiama **opposto** con la notazione additiva ($-a$, neutro 0) e **inverso** con quella moltiplicativa (a^{-1}, neutro 1). Nella tabella: l'operazione è commutativa se la tabella è simmetrica rispetto alla diagonale; e è neutro se la sua riga e la sua colonna ripetono l'intestazione; a ha un simmetrico se nella sua riga compare e.

2.2 Semigruppi, monoidi e gruppi

Un insieme con una o più operazioni è una **struttura algebrica**. Con una sola operazione interna $(A, *)$:

Struttura	Richiede
semigruppo	$*$ associativa
monoide	semigruppo con elemento neutro
gruppo	monoide in cui ogni elemento ha il simmetrico
gruppo abeliano	gruppo con $*$ commutativa

Ogni riga della tabella aggiunge una richiesta alla precedente. Gli esempi da sapere:

- $(\mathbb{N}, +)$ è un semigruppo ma non un monoide: lo 0 non c'è.
- $(\mathbb{N}_0, +)$ è un monoide ma non un gruppo: nessun numero positivo ha l'opposto in $\mathbb{N}_0$.

- $(\mathbb{Z}, +)$, $(\mathbb{Q}, +)$, $(\mathbb{R}, +)$ sono gruppi abeliani; $(\mathbb{Q} \setminus \{0\}, \cdot)$ anche.
- $(\mathbb{Z}, \cdot)$ è un monoide commutativo (neutro 1), non un gruppo: 2 non ha inverso intero.
- $(\mathbb{Z}, -)$ non è neppure un semigruppo: $(5 - 3) - 1 \neq 5 - (3 - 1)$.
- Le **stringhe** su un alfabeto con la **concatenazione** formano un monoide non commutativo: il neutro è la stringa vuota, e `"ab" + "c"` $\neq$ `"c" + "ab"`.
- $(\mathcal{P}(S), \cup)$ è un monoide commutativo con neutro $\emptyset$; $(\mathcal{P}(S), \cap)$ con neutro S; $(\mathcal{P}(S), \Delta)$ è un gruppo abeliano: neutro $\emptyset$, e ogni X è il simmetrico di sé stesso, perché $X \,\Delta\, X = \emptyset$.
- Le biiezioni di un insieme in sé con la composizione formano un gruppo, non abeliano se l'insieme ha almeno tre elementi.

I resti modulo n. $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$, con la somma e il prodotto "modulo n": si fa l'operazione e si prende il resto della divisione per n (sono le classi della congruenza del modulo 1). $(\mathbb{Z}_n, +_n)$ è un gruppo abeliano: l'opposto di $a \neq 0$ è $n - a$. È l'aritmetica degli interi senza segno di un calcolatore a k bit, che lavora modulo 2^k: `255 + 1` in un byte dà 0.

2.3 Anelli, corpi e campi

Con due operazioni $(A, \oplus, \otimes)$ si ha un **anello** se $(A, \oplus)$ è un gruppo abeliano e $\otimes$ è associativa e distributiva rispetto a $\oplus$:

$$a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c), \qquad (b \oplus c) \otimes a = (b \otimes a) \oplus (c \otimes a).$$

L'anello è **unitario** se $\otimes$ ha l'elemento neutro, **commutativo** se $\otimes$ è commutativa. $(\mathbb{Z}, +, \cdot)$ è un anello commutativo unitario, e così $(\mathcal{P}(S), \Delta, \cap)$, con l'unità S.

Un anello unitario con più di un elemento in cui ogni elemento diverso dal neutro di $\oplus$ ha l'inverso rispetto a $\otimes$ è un **corpo**; un corpo commutativo è un **campo**. $(\mathbb{Q}, +, \cdot)$, $(\mathbb{R}, +, \cdot)$ e $(\mathbb{C}, +, \cdot)$ sono campi; $(\mathbb{Z}, +, \cdot)$ no.

$(\mathbb{Z}_n, +_n, \cdot_n)$ è sempre un anello commutativo unitario, ed è un campo **se e solo se n è primo**. Se n non è primo, $n = a \cdot b$ con $1 < a, b < n$ e $a \cdot_n b = 0$: due elementi non nulli hanno prodotto nullo (**divisori dello zero**), e a non può avere inverso. In $\mathbb{Z}_6$, $2 \cdot 3 = 0$; in $\mathbb{Z}_7$ invece ogni elemento non nullo è invertibile: $1 \cdot 1 = 2 \cdot 4 = 3 \cdot 5 = 6 \cdot 6 = 1$.

2.4 Contare: i due principi

Principio della somma. Se una scelta si fa *o* in un insieme di m modi *o* in un altro, disgiunto, di n modi, i modi sono $m + n$. È $|A \cup B| = |A| + |B|$ per insiemi disgiunti.

Principio del prodotto. Se una scelta si fa in due fasi, la prima in m modi e la seconda in n modi qualunque sia stata la prima, i modi sono $m \cdot n$. È $|A \times B| = |A| \cdot |B|$, e si estende a k fasi.

Quasi tutto il calcolo combinatorio viene da questi due principi. Per scegliere la formula bastano due domande sul problema: **conta l'ordine?** (una sequenza o un insieme?) e **sono ammesse le ripetizioni?** (lo stesso oggetto può essere scelto più volte?).

2.5 Disposizioni, permutazioni, combinazioni

Si scelgono k oggetti da un insieme di n. Il **fattoriale** è $n! = n(n-1)\cdots 2 \cdot 1$, con $0! = 1$.

Disposizioni semplici (conta l'ordine, niente ripetizioni): sequenze di k oggetti distinti. Il primo si sceglie in n modi, il secondo in $n - 1$, ..., il k-esimo in $n - k + 1$:

$$D_{n,k} = n(n-1)\cdots(n-k+1) = \frac{n!}{(n-k)!}.$$

Disposizioni con ripetizione (conta l'ordine, ripetizioni ammesse): ogni posizione si sceglie in n modi, $D'_{n,k} = n^k$. Sono le stringhe di lunghezza k su un alfabeto di n simboli: $2^8 = 256$ byte diversi.

Permutazioni: le disposizioni semplici di tutti gli n oggetti, cioè i modi di metterli in fila, $P_n = n!$. Se fra gli n oggetti ce ne sono k_1 uguali fra loro, k_2 uguali fra loro, e così via, scambiare oggetti uguali non dà una fila

nuova: le **permutazioni con ripetizione** sono

$$P_n^{(k_1,k_2,\dots)} = \frac{n!}{k_1!\,k_2!\cdots}.$$

Combinazioni semplici (l'ordine non conta, niente ripetizioni): sottoinsiemi di k elementi. Ogni sottoinsieme corrisponde a $k!$ disposizioni (i suoi ordinamenti), quindi

$$C_{n,k} = \frac{D_{n,k}}{k!} = \frac{n!}{k!\,(n-k)!} = \binom{n}{k}.$$

Combinazioni con ripetizione (l'ordine non conta, ripetizioni ammesse): gruppi di k oggetti in cui lo stesso può comparire più volte, come scegliere k palline da n colori. Sono

$$C'_{n,k} = \binom{n+k-1}{k}.$$

Il motivo: un gruppo si descrive con k "stelle" (gli oggetti) e $n-1$ "sbarre" (i separatori fra un tipo e l'altro), e basta scegliere quali delle $n+k-1$ posizioni sono stelle.

	Niente ripetizioni	**Con ripetizioni**
conta l'ordine	$D_{n,k} = \dfrac{n!}{(n-k)!}$	n^k
non conta l'ordine	$\binom{n}{k}$	$\binom{n+k-1}{k}$

2.6 I coefficienti binomiali

I numeri $\binom{n}{k}$ hanno proprietà che semplificano i conti:

$$\binom{n}{k} = \binom{n}{n-k}, \qquad \binom{n}{0} = \binom{n}{n} = 1, \qquad \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

La prima dice che scegliere i k elementi da prendere equivale a scegliere gli $n-k$ da lasciare. L'ultima (formula di Stifel) si legge così: fissato un elemento, i sottoinsiemi di k elementi o lo contengono (e si scelgono gli altri $k-1$ fra $n-1$) o non lo contengono (e si scelgono tutti fra $n-1$). È la regola del **triangolo di Tartaglia**, in cui ogni numero è la somma dei due sopra.

Binomio di Newton

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k}.$$

Sviluppando il prodotto di n fattori $(a+b)$, il termine $a^k b^{n-k}$ compare tante volte quanti sono i modi di scegliere da quali k fattori prendere a. Con $a=b=1$ si ottiene $\sum_k \binom{n}{k} = 2^n$: i sottoinsiemi di un insieme di n elementi, contati per numero di elementi, sono 2^n, come nel modulo 1.

Esempi ed esercizi svolti

Aggiunta in revisione · fonte: esercizi nuovi sul modello delle prove 2016-2020 e del foglio `esercizi_matematica`; risultati controllati con `strumenti/verifica.py` (FI-03, FI-04)

Esercizio 1. Che struttura è?

Struttura	Risposta	Perché
$(\mathbb{N}_0, \max)$	monoide commutativo	max è associativa e commutativa, neutro 0; $\max(3,x)=0$ non ha soluzione

Struttura	Risposta	Perché
$(\mathbb{Z}, *)$ con $a * b = a + b + 1$	gruppo abeliano	associativa; neutro -1 ($a + (-1) + 1 = a$); simmetrico di a è $-a - 2$
$(\mathbb{Z}_6, \cdot_6)$	monoide commutativo, non gruppo	neutro 1; 0, 2, 3, 4 non hanno inverso
$(\mathbb{Z}_5 \setminus \{0\}, \cdot_5)$	gruppo abeliano	5 è primo: $1 \cdot 1 = 2 \cdot 3 = 4 \cdot 4 = 1$
$(\mathbb{R}, *)$ con $a * b = (a + b)/2$	nessuna delle quattro	commutativa ma non associativa: $(0 * 0) * 4 = 2$, $0 * (0 * 4) = 1$

Esercizio 2. Una tabella. Su $A = \{e, a, b\}$:

$*$	e	a	b
e	e	a	b
a	a	b	e
b	b	e	a

La tabella è simmetrica (commutativa), e è neutro, e e compare in ogni riga: a e b sono l'uno il simmetrico dell'altro. L'associatività si controlla sui casi (sono 27, ma con il neutro bastano quelli con a e b): per esempio $(a * a) * b = b * b = a$ e $a * (a * b) = a * e = a$. È un gruppo abeliano: è $\mathbb{Z}_3$ con $e = 0$, $a = 1$, $b = 2$.

Esercizio 3. Inversi modulo n. Trovare gli inversi in $\mathbb{Z}_7$ e dire perché $\mathbb{Z}_8$ non è un campo.

In $\mathbb{Z}_7$: $1^{-1} = 1$, $2^{-1} = 4$, $3^{-1} = 5$, $4^{-1} = 2$, $5^{-1} = 3$, $6^{-1} = 6$. In $\mathbb{Z}_8$: $2 \cdot 4 = 8 \equiv 0$, quindi 2 è un divisore dello zero; se avesse un inverso x, da $2 \cdot 4 = 0$ si avrebbe $4 = x \cdot 2 \cdot 4 = 0$, assurdo.

Esercizio 4. La gelateria (sul modello dell'esercizio delle crociere). Ci sono 12 gusti.

1. Coppette con 3 gusti diversi, l'ordine non conta: $\binom{12}{3} = 220$.
2. Coni a 3 strati di gusti diversi, conta l'ordine dal basso: $D_{12,3} = 12 \cdot 11 \cdot 10 = 1320$.
3. Coppette con 3 palline, anche dello stesso gusto: $\binom{14}{3} = 364$.
4. Coni a 3 strati con gusti anche ripetuti: $12^3 = 1728$.
5. Coppette da 2 gusti diversi oppure coni da 2 strati di gusti diversi: $\binom{12}{2} + D_{12,2} = 66 + 132 = 198$ (principio della somma).

Esercizio 5. Password. Password di 8 caratteri fra 26 lettere minuscole e 10 cifre. Quante sono? Quante contengono almeno una cifra?

Con ripetizione e conta l'ordine: $36^8 = 2\,821\,109\,907\,456$. Quelle senza cifre sono $26^8 = 208\,827\,064\,576$; quelle con almeno una cifra sono la differenza, $2\,612\,282\,842\,880$. Contare "almeno uno" passando al complementare evita di dividere in casi.

Esercizio 6. Anagrammi. Quanti anagrammi (anche senza senso) ha INFORMATICA?

11 lettere, con I e A due volte ciascuna: $\dfrac{11!}{2!\,2!} = \dfrac{39\,916\,800}{4} = 9\,979\,200$.

Esercizio 7. Commissioni. Fra 8 uomini e 6 donne si sceglie una commissione di 5 persone.

Con esattamente 2 donne: $\binom{6}{2}\binom{8}{3} = 15 \cdot 56 = 840$. Con almeno una donna: tutte meno quelle di soli uomini, $\binom{14}{5} - \binom{8}{5} = 2002 - 56 = 1946$.

Esercizio 8. Byte e binomio. Quanti byte hanno esattamente tre bit a 1? Qual è il coefficiente di x^3 in $(x + 2)^5$?

Si scelgono le 3 posizioni dei bit a 1 fra 8: $\binom{8}{3} = 56$. Nel binomio, il termine con x^3 è $\binom{5}{3} x^3 2^2 = 40x^3$.

Esercizio 9. Inclusione-esclusione. Quanti numeri da 1 a 100 sono divisibili per 2, 3 o 5?

Divisibili per 2: 50; per 3: 33; per 5: 20; per 6: 16; per 10: 10; per 15: 6; per 30: 3. Totale $50 + 33 + 20 - 16 - 10 - 6 + 3 = 74$.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; risultati controllati con `strumenti/verifica.py` (FI-03, FI-04)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. Scegliere la formula. Una squadra di calcetto: fra 9 giocatori se ne scelgono 5.

Passo 1: i 5 titolari, senza ruoli. L'ordine non conta, niente ripetizioni: $\binom{9}{5} = 126$.

Passo 2: con 5 ruoli diversi (portiere, due difensori distinti, due attaccanti distinti). Ora conta a chi va quale ruolo: $D_{9,5} = 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 = 15\,120$.

Passo 3: un portiere e 4 giocatori di movimento senza ruoli. Due fasi: il portiere fra 9, poi 4 fra gli 8 rimasti, $9 \cdot \binom{8}{4} = 9 \cdot 70 = 630$.

Passo 4: il controllo. $630 = 5 \cdot 126$: ogni gruppo di 5 titolari dà 5 squadre diverse a seconda di chi va in porta.

Guidato 2. La tabella di $\mathbb{Z}_6$ per il prodotto.

Passo 1: la tabella delle righe utili. Riga del 2: $0, 2, 4, 0, 2, 4$. Riga del 3: $0, 3, 0, 3, 0, 3$. Riga del 5: $0, 5, 4, 3, 2, 1$.

Passo 2: gli inversi. Un elemento è invertibile se nella sua riga compare 1: solo 1 e 5 ($5 \cdot 5 = 25 \equiv 1$).

Passo 3: i divisori dello zero. Nelle righe di 2, 3 e 4 compare 0 fuori dalla colonna dello 0: $2 \cdot 3 = 3 \cdot 4 = 0$.

Passo 4: la conclusione. $\mathbb{Z}_6$ è un anello commutativo unitario ma non un campo. Gli elementi invertibili sono quelli primi con 6.

Guidato 3. Mani di carte. Da un mazzo di 40 carte (4 semi da 10) si pescano 4 carte.

Passo 1: le mani possibili. $\binom{40}{4} = 91\,390$.

Passo 2: mani con 4 carte dello stesso seme. Si sceglie il seme (4 modi), poi 4 carte su 10: $4 \cdot \binom{10}{4} = 4 \cdot 210 = 840$.

Passo 3: mani con un seme per carta. Una carta per seme: $10^4 = 10\,000$.

Passo 4: la probabilità, se servisse: casi favorevoli su casi possibili, $840 / 91\,390 \approx 0{,}9\%$.

Riepilogo

Un'operazione interna associa a due elementi di un insieme un elemento dello stesso insieme; le proprietà associativa, commutativa, l'esistenza del neutro e dei simmetrici definiscono semigruppi, monoidi, gruppi e gruppi abeliani. Con due operazioni si hanno anelli (gruppo abeliano più un'operazione associativa e distributiva), corpi e campi; $\mathbb{Z}_n$ è un campo solo per n primo, altrimenti ha divisori dello zero. Per contare si usano i principi della somma e del prodotto; la formula si sceglie chiedendosi se conta l'ordine e se sono ammesse ripetizioni: disposizioni semplici e con ripetizione, permutazioni (anche con oggetti ripetuti), combinazioni semplici e con ripetizione. I coefficienti binomiali sono simmetrici, seguono il triangolo di Tartaglia e danno lo sviluppo del binomio di Newton.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (FI-03, FI-04)**

Strutture

Struttura	Associativa	Neutro	Simmetrici	Commutativa
semigruppo	sì			
monoide	sì	sì		
gruppo	sì	sì	sì	

Struttura	Associativa	Neutro	Simmetrici	Commutativa
gruppo abeliano	sì	sì	sì	sì

Anello: $(A, \oplus)$ gruppo abeliano, $\otimes$ associativa e distributiva. Corpo: anello unitario, inversi per tutti tranne lo zero. Campo: corpo commutativo. $\mathbb{Z}_n$ campo $\iff$ n primo.

Combinatoria

$$D_{n,k} = \frac{n!}{(n-k)!}, \quad D'_{n,k} = n^k, \quad P_n = n!, \quad \frac{n!}{k_1!k_2!\dots}, \quad \binom{n}{k} = \frac{n!}{k!(n-k)!}, \quad C'_{n,k} = \binom{n+k-1}{k}.$$

$$\binom{n}{k} = \binom{n}{n-k}, \qquad \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}, \qquad (a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k}, \qquad \sum_{k=0}^{n} \binom{n}{k} = 2^n.$$

Termini per l’orale

Termine	In una riga
Operazione interna	$A \times A \to A$, il risultato resta in A
Elemento neutro, simmetrico	non cambia gli altri; riporta al neutro
Divisore dello zero	$a, b \neq 0$ con $ab = 0$
Principio del prodotto	scelte in fasi: si moltiplicano i modi
Disposizione	sequenza ordinata di k oggetti
Combinazione	sottoinsieme di k oggetti
Permutazione con ripetizione	ordinamenti di oggetti non tutti distinti

Domande che tornano spesso

- Definire semigruppo, monoide, gruppo, anello, campo, con esempi e controesempi.
- Perché $(\mathbb{N}_0, +)$ non è un gruppo? Perché $\mathbb{Z}_6$ non è un campo?
- Come si sceglie fra disposizioni, permutazioni e combinazioni? Ricavare $\binom{n}{k}$ dalle disposizioni.
- Enunciare il binomio di Newton e la formula di Stifel.

3 · Calcolatori, algoritmi e primi programmi in C++

In questo modulo. Si usano le funzioni e la logica del modulo 1. Si parte da com'è fatto un calcolatore e da come rappresenta numeri e caratteri con i bit. Poi si vede che cos'è un algoritmo e come un programma scritto in un linguaggio ad alto livello diventa un eseguibile. Il resto del modulo è il primo pezzo di C++: struttura di un programma, ingresso e uscita, variabili e tipi, operatori ed espressioni, istruzioni condizionali e iterative. Alla fine del modulo si sa scrivere, compilare e provare un programma con letture da tastiera, condizioni e cicli, valutare a mano un'espressione C++ e scrivere la condizione che corrisponde a un intervallo, come negli esercizi della prova di teoria.

> **Aggiunta in revisione · fonte: slide del corso (2.1 Introduzione ai calcolatori, 2.2 Introduzione alla programmazione), esercitazioni E00-E03; H.M. Deitel e P.J. Deitel, *C++. Fondamenti di programmazione*, Apogeo (FI-05, FI-06)**
>
> Il modulo è scritto in revisione dal materiale del corso. Tutti i programmi sono stati compilati con `g++ -std=c++17 -Wall -Wextra` ed eseguiti con gli ingressi indicati da `strumenti/verifica.py`; la prima riga di ogni programma (`// file: …`) è il nome del file.

3.1 Il calcolatore

Un calcolatore esegue istruzioni su dati, entrambi conservati in memoria. Il modello è quello di **von Neumann**: una **CPU** (unità centrale di elaborazione) con l'**unità di controllo**, che legge le istruzioni dalla memoria e le fa eseguire una alla volta, e l'**ALU** (unità aritmetico-logica), che fa i calcoli e i confronti; una **memoria centrale** (la RAM, veloce, volatile) che contiene programmi e dati in uso; le **memorie di massa** (dischi, SSD) che conservano i dati in modo permanente; le **periferiche** di ingresso e uscita (tastiera, schermo, rete); i **bus**, i collegamenti fra le parti. Dentro la CPU ci sono i **registri**, poche celle velocissime. Il ciclo della CPU è sempre lo stesso: preleva l'istruzione, la decodifica, la esegue, passa alla successiva.

L'**hardware** è la parte fisica, il **software** l'insieme dei programmi: il **sistema operativo** gestisce le risorse (processore, memoria, file, periferiche) e offre servizi ai programmi applicativi.

3.2 Rappresentare l'informazione

Tutto in un calcolatore è fatto di **bit** (*binary digit*), che valgono 0 o 1. Il raggruppamento più noto è il **byte**, 8 bit; poi kilobyte, megabyte, gigabyte. Con n bit si rappresentano 2^n valori diversi (modulo 2): un byte ha 256 valori.

Numeri naturali in base 2. Come in base 10 ogni cifra vale una potenza di 10, in base 2 ogni bit vale una potenza di 2: $1011_2 = 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 11$. Per passare da decimale a binario si divide ripetutamente per 2 e si leggono i resti dall'ultimo al primo: $11 \to$ resti $1, 1, 0, 1 \to 1011$. La base 16 (**esadecimale**, cifre 0-9 e A-F) è un modo compatto di scrivere il binario: ogni cifra esadecimale corrisponde a 4 bit, $1011\,0110_2 = \texttt{B6}_{16} = 182$.

Interi con segno. Si usa il **complemento a 2**: con n bit si rappresentano i numeri da -2^{n-1} a $2^{n-1} - 1$, il bit più significativo indica il segno, e per cambiare segno si invertono tutti i bit e si aggiunge 1. Così la somma si fa con lo stesso circuito per positivi e negativi. Un `int` di 32 bit va da $-2\,147\,483\,648$ a $2\,147\,483\,647$: oltre si ha **overflow**, e il risultato "gira".

Numeri reali. Si usa la **virgola mobile**: segno, mantissa ed esponente, come nella notazione scientifica. I reali rappresentabili sono un numero finito, quindi quasi sempre i calcoli sono approssimati: `0.1 + 0.2` non è esattamente `0.3`.

Caratteri. A ogni carattere corrisponde un numero, secondo il codice **ASCII** (7 bit, 128 caratteri; le estensioni come Unicode ne hanno molti di più). Da ricordare: `'0'` = 48, `'A'` = 65, `'a'` = 97, e le cifre, le maiuscole e le minuscole sono consecutive. Per questo `c - '0'` dà il valore della cifra `c`, e `c - 'a' + 'A'` la maiuscola di una minuscola.

3.3 Algoritmi, programmi, linguaggi

Un **algoritmo** è una sequenza finita di passi non ambigui ed eseguibili che risolve un problema: deve terminare, ogni passo deve essere chiaro, e deve funzionare per tutti i dati previsti. Un **programma** è un algoritmo scritto in un **linguaggio di programmazione**. Il processore capisce solo il **linguaggio macchina** (istruzioni codificate in binario); l'**assembly** ne è la versione simbolica; i **linguaggi ad alto livello** come il C++ sono vicini al modo di ragionare del programmatore e indipendenti dalla macchina.

Il passaggio da sorgente a eseguibile: il **preprocessore** tratta le direttive `#include` (inserisce i file di intestazione); il **compilatore** traduce il sorgente in codice oggetto e segnala gli errori di sintassi; il **linker** collega il codice oggetto con le librerie e produce l'eseguibile. Gli errori sono di tre tipi: **di compilazione** (sintassi, tipi: il programma non parte), **di esecuzione** (divisione per zero, accesso fuori da un array: il programma si interrompe o si comporta in modo imprevedibile), **logici** (il programma gira ma dà risultati sbagliati: si trovano solo provandolo).

La **programmazione strutturata** costruisce ogni algoritmo con tre schemi: **sequenza** (un'istruzione dopo l'altra), **selezione** (scegliere fra alternative con una condizione), **iterazione** (ripetere finché vale una condizione). Il teorema di Böhm e Jacopini dice che bastano questi tre per scrivere qualunque algoritmo, senza salti arbitrari.

3.4 Il primo programma: variabili e tipi

```cpp
// file: somma.cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;                       // due variabili intere
    cout << "Inserisci due interi: ";
    cin >> a >> b;                  // lettura da tastiera
    int s = a + b;                  // dichiarazione con inizializzazione
    cout << "La somma e' " << s << endl;
    return 0;                       // 0 = terminazione regolare
}
```

`#include <iostream>` rende disponibili `cin` (il flusso d'ingresso, la tastiera) e `cout` (il flusso d'uscita, lo schermo); `using namespace std;` evita di scrivere `std::cout`. L'esecuzione comincia da `main`, che restituisce un intero al sistema operativo. Ogni istruzione finisce con `;`, i blocchi stanno fra graffe, `//` apre un commento fino alla fine della riga. `<<` "manda" valori a `cout`, `>>` "estrae" valori da `cin` saltando gli spazi; `endl` va a capo.

Una **variabile** è una zona di memoria con un **nome**, un **tipo** e un **valore**. Il tipo dice quanta memoria serve, quali valori sono ammessi e quali operazioni si possono fare. I tipi fondamentali:

Tipo	Valori	Esempio
`int`	interi (di solito 32 bit)	`int n = -7;`
`double`, `float`	reali in virgola mobile (doppia, singola precisione)	`double x = 3.14;`
`char`	un carattere (un byte, il suo codice)	`char c = 'A';`
`bool`	`true` o `false`	`bool trovato = false;`

Il nome (identificatore) è fatto di lettere, cifre e `_`, non comincia con una cifra e non è una parola riservata; maiuscole e minuscole sono diverse. Una variabile va **dichiarata** prima dell'uso; se non viene inizializzata il suo valore è indefinito (quello che c'era in memoria). Una **costante** si dichiara con `const` e non può cambiare: `const double PI = 3.14159;`.

3.5 Operatori ed espressioni

Aritmetici: `+`, `-`, `*`, `/`, `%`. Fra due interi `/` è la **divisione intera** (troncata verso lo zero) e `%` il resto: `7 / 2` vale 3, `7 % 2` vale 1, `-7 / 2` vale -3. Se almeno un operando è reale la divisione è reale: `7 / 2.0` vale 3.5. Attenzione a `double m = (a + b) / 2;` con `a` e `b` interi: la divisione è intera e solo dopo il risultato diventa `double`. Si corregge con una **conversione esplicita**, `static_cast<double>(a + b) / 2`, o scrivendo `2.0`.

Assegnamento: `x = espr` calcola l'espressione e la mette in `x`; è anch'esso un'espressione, che vale il valore assegnato. Forme composte: `x += 3` equivale a `x = x + 3`, e così `-=`, `*=`, `/=`, `%=`. **Incremento e decremento**: `++x` incrementa e poi vale il nuovo valore (prefisso), `x++` vale il vecchio valore e poi incrementa (postfisso).

Relazionali: `==`, `!=`, `<`, `<=`, `>`, `>=`; **logici**: `&&` (e), `||` (o), `!` (non). Il risultato è un `bool`; in C++ ogni valore numerico diverso da 0 conta come vero e 0 come falso. Gli operatori logici sono valutati in **cortocircuito**: in `a && b`, se `a` è falso `b` non viene valutato; in `a || b`, se `a` è vero neanche. Si sfrutta per scrivere `(d != 0) && (n / d > 2)` senza rischiare la divisione per zero.

Un errore classico è scrivere `if (x = 5)` invece di `if (x == 5)`: è un assegnamento, vale 5, cioè sempre vero. E una condizione come $0 < x < 10$ va scritta `(x > 0) && (x < 10)`: `0 < x < 10` si compila ma confronta con 10 il risultato (0 o 1) di `0 < x`.

Precedenza (dalla più alta): `!`, `++`, `--` unari; `*`, `/`, `%`; `+`, `-`; `<`, `<=`, `>`, `>=`; `==`, `!=`; `&&`; `||`; assegnamenti. Nel dubbio, parentesi.

3.6 Le istruzioni condizionali

```cpp
// file: giudizio.cpp
#include <iostream>
using namespace std;

int main() {
    int voto;
    cin >> voto;
    if (voto < 0 || voto > 30)
        cout << "voto non valido" << endl;
    else if (voto < 18)
        cout << "insufficiente" << endl;
    else if (voto < 24)
        cout << "sufficiente" << endl;
    else if (voto < 28)
        cout << "buono" << endl;
    else
        cout << "ottimo" << endl;
    return 0;
}
```

`if (condizione) istruzione1 else istruzione2` esegue la prima se la condizione è vera, la seconda altrimenti; il ramo `else` è facoltativo, e se un ramo ha più istruzioni le si chiude in un blocco `{ }`. Nella catena `else if` le condizioni si controllano in ordine e si esegue solo il primo ramo vero: per questo il secondo test può limitarsi a `voto < 24`, sapendo già che `voto >= 18`. Un `else` si lega sempre all'`if` più vicino senza `else`: con gli `if` annidati conviene usare le graffe.

L'istruzione **`switch`** sceglie in base al valore di un'espressione intera o carattere:

```cpp
// file: calcolatrice.cpp
#include <iostream>
using namespace std;

int main() {
    double x, y;
    char op;
    cin >> x >> op >> y;
    switch (op) {
```

```
            case '+': cout << x + y << endl; break;
            case '-': cout << x - y << endl; break;
            case '*':
            case 'x': cout << x * y << endl; break;    // due etichette
            case '/':
                if (y != 0) cout << x / y << endl;
                else cout << "divisione per zero" << endl;
                break;
            default: cout << "operatore sconosciuto" << endl;
        }
        return 0;
    }
```

L'esecuzione salta all'etichetta `case` uguale al valore e prosegue **fino al `break`**: senza `break` si "cade" nei casi successivi, cosa che si sfrutta per dare più etichette allo stesso codice. `default` raccoglie i valori non previsti.

3.7 Le istruzioni iterative

C++ ha tre cicli. **`while (condizione) corpo`**: controlla la condizione prima di ogni ripetizione, quindi il corpo può non essere mai eseguito. **`do corpo while (condizione);`**: controlla dopo, quindi il corpo si esegue almeno una volta; è il ciclo giusto per **validare** un dato letto, che va chiesto di nuovo finché non è valido. **`for (inizializzazione; condizione; aggiornamento) corpo`**: equivale a un `while` con le tre parti raccolte in testa, ed è il ciclo naturale quando il numero di ripetizioni è controllato da un **contatore** (cicli *definiti*). Quando invece si ripete finché succede qualcosa (un valore **sentinella**, un dato non valido) il ciclo è *indefinito*.

`break` esce subito dal ciclo più interno; `continue` salta il resto del corpo e passa alla ripetizione successiva.

```
// file: media.cpp
#include <iostream>
using namespace std;

int main() {
    int x, n = 0, somma = 0;
    cin >> x;
    while (x != -1) {              // -1 e' la sentinella
        if (x < 0) {               // altri negativi: scartati
            cin >> x;
            continue;
        }
        somma += x;
        n++;
        cin >> x;
    }
    if (n > 0)
        cout << "media " << static_cast<double>(somma) / n << endl;
    else
        cout << "nessun valore" << endl;
    return 0;
}
```

```
// file: validazione.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    do {
        cout << "Numero fra 1 e 10: ";
        cin >> n;
```

```
    } while (n < 1 || n > 10);
    for (int i = 1; i <= 10; i++)
        cout << n << " x " << i << " = " << n * i << endl;
    return 0;
}
```

Una variabile dichiarata nell'intestazione del `for`, come `i`, esiste solo dentro il ciclo. Per scrivere un ciclo corretto conviene chiedersi tre cose: da dove parte (inizializzazione), quando si ferma (condizione, che prima o poi deve diventare falsa, altrimenti il ciclo è **infinito**), che cosa cambia a ogni giro.

Esempi ed esercizi svolti

> **Aggiunta in revisione · fonte: esercizi nuovi sul modello delle prove 2016-2020 e delle esercitazioni E01-E03; programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-05, FI-06)**

Esercizio 1. Conversioni. Scrivere 45 in binario e in esadecimale; scrivere -45 in complemento a 2 su 8 bit; che carattere ha codice 100?

$45 = 32 + 8 + 4 + 1 = 0010\,1101_2 = 2\mathrm{D}_{16}$. Per -45: si invertono i bit, $1101\,0010$, e si aggiunge 1: $1101\,0011$. Controllo: $-128 + 64 + 16 + 2 + 1 = -45$. Il codice 100 è `'a' + 3`, cioè `'d'`.

Esercizio 2. Valutare espressioni. Con `int a = 7, b = 2;` e `double c;`, che valore hanno, nell'ordine, queste espressioni (ognuna parte dai valori lasciati dalla precedente)?

```
// file: espressioni.cpp
#include <iostream>
using namespace std;

int main() {
    int a = 7, b = 2, zero = 0;
    double c;
    cout << a / b << " " << a % b << endl;       // 3 1
    c = a / b;
    cout << c << " " << a / (double)b << endl;   // 3 3.5
    int d = a++ + ++b;                           // 7 + 3
    cout << d << " " << a << " " << b << endl;   // 10 8 3
    cout << (a > b && b == 3) << " " << !(a % 2) << endl;   // 1 1
    a -= b * 2;                                  // 8 - 6
    cout << a << " " << (a == 2 || 1 / zero > 0) << endl;
    return 0;
}
```

`c = a / b` vale 3 perché la divisione è fra interi; `a / (double)b` è 3.5. In `a++ + ++b` si usa il vecchio `a` (7) e il nuovo `b` (3). `!(a % 2)` è vero perché `a` vale 8. Nell'ultima riga `a` vale 2, `a == 2` è vero e il cortocircuito evita di valutare `1 / zero`, che farebbe fermare il programma: stampa `2 1`.

Esercizio 3. Intervalli (sul modello della prova intermedia). Completare le espressioni in modo che `y` sia vero se e solo se l'intero `x` appartiene all'insieme.

Insieme	Espressione
$[-5, 5]$	`y = (x >= -5) && (x <= 5);`
$(-\infty, 0) \cup (10, 20]$	`y = (x < 0) || ((x > 10) && (x <= 20));`
$[0, 100]$, $x \neq 50$, x pari	`y = (x >= 0) && (x <= 100) && (x != 50) && (x % 2 == 0);`
$\mathbb{Z} \setminus [3, 7]$	`y = (x < 3) || (x > 7);` oppure `y = !((x >= 3) && (x <= 7));`

L'ultima riga mostra De Morgan: il complementare di un intervallo è un "o" di due condizioni.

```cpp
// file: intervalli.cpp
#include <iostream>
using namespace std;

int main() {      // conta gli interi di [-30, 120] in ciascun insieme
    int c1 = 0, c2 = 0, c3 = 0, c4 = 0;
    for (int x = -30; x <= 120; x++) {
        if ((x >= -5) && (x <= 5)) c1++;
        if ((x < 0) || ((x > 10) && (x <= 20))) c2++;
        if ((x >= 0) && (x <= 100) && (x != 50) && (x % 2 == 0)) c3++;
        if (!((x >= 3) && (x <= 7))) c4++;
    }
    cout << c1 << " " << c2 << " " << c3 << " " << c4 << endl;
    return 0;
}
```

Il programma stampa `11 40 50 146`: 11 interi in $[-5, 5]$; 30 negativi più 10 in $(10, 20]$; i 51 pari da 0 a 100 meno il 50; tutti i 151 meno i 5 di $[3, 7]$.

Esercizio 4. Somma delle cifre. Leggere un intero positivo e stampare la somma delle sue cifre.

```cpp
// file: cifre.cpp
#include <iostream>
using namespace std;

int main() {
    int n, somma = 0;
    cin >> n;
    while (n > 0) {
        somma += n % 10;    // l'ultima cifra
        n /= 10;            // la si toglie
    }
    cout << somma << endl;
    return 0;
}
```

Con 4096 stampa 19. `n % 10` e `n / 10` sono i due attrezzi per lavorare sulle cifre di un intero.

Esercizio 5. Massimo comun divisore con l'algoritmo di Euclide: $\mathrm{MCD}(a, b) = \mathrm{MCD}(b, a \bmod b)$, e $\mathrm{MCD}(a, 0) = a$.

```cpp
// file: mcd.cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    cout << a << endl;
    return 0;
}
```

Con 84 e 36: $(84, 36) \to (36, 12) \to (12, 0)$, stampa 12. Il ciclo termina perché il resto diminuisce strettamente a ogni passo.

Esercizio 6. Numero primo. Dire se un intero $n \geq 2$ è primo.

```cpp
// file: primo.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    bool primo = (n >= 2);
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) {
            primo = false;
            break;          // inutile continuare
        }
    cout << n << (primo ? " e' primo" : " non e' primo") << endl;
    return 0;
}
```

Basta provare i divisori fino a $\sqrt{n}$: se $n = d \cdot e$ con $d \leq e$, allora $d \leq \sqrt{n}$. L'operatore `c ? x : y` vale `x` se `c` è vero, `y` altrimenti.

Esercizio 7. Da decimale a binario. Stampare i bit di un intero fra 0 e 255, dal più significativo.

```cpp
// file: binario.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int p = 128; p >= 1; p /= 2) {   // potenze di 2 decrescenti
        if (n >= p) {
            cout << 1;
            n -= p;
        } else
            cout << 0;
    }
    cout << endl;
    return 0;
}
```

Con 45 stampa `00101101`. Invece delle divisioni successive (che danno i bit dal meno significativo, quindi al contrario) si confronta il numero con le potenze di 2 dalla più grande.

Esercizio 8. Cicli annidati. Stampare un triangolo di asterischi alto h.

```cpp
// file: triangolo.cpp
#include <iostream>
using namespace std;

int main() {
    int h;
    cin >> h;
    for (int i = 1; i <= h; i++) {
        for (int j = 1; j <= h - i; j++) cout << ' ';
        for (int j = 1; j <= 2 * i - 1; j++) cout << '*';
        cout << endl;
    }
```

```
    return 0;
}
```

La riga i ha $h - i$ spazi e $2i - 1$ asterischi: prima si scrive la formula di una riga, poi il ciclo esterno sulle righe.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-05, FI-06)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. Che cosa stampa? Seguire a mano il programma con l'ingresso 5.

```
// file: traccia.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int i = 0, s = 0;
    while (i < n) {
        i++;
        if (i % 2 == 0) continue;
        s += i * i;
        if (s > 20) break;
    }
    cout << i << " " << s << endl;
    cout << (s > 30 || n / (i - 5) > 0) << endl;
    return 0;
}
```

Passo 1: la tabella dei valori. Si scrive una riga per giro con `i` e `s` dopo il corpo: $i = 1$, $s = 1$; $i = 2$, pari, `continue`, $s = 1$; $i = 3$, $s = 10$; $i = 4$, `continue`; $i = 5$, $s = 35$, e $35 > 20$: `break`.

Passo 2: la prima riga. `5 35`.

Passo 3: la seconda riga. `s > 30` è vero: per il cortocircuito `n / (i - 5)`, che sarebbe una divisione per zero, non si valuta; stampa `1` (un `bool` vero si stampa come 1).

Guidato 2. Validare e contare. Leggere numeri fino a 0; contare quanti sono positivi, quanti negativi, e trovare il più grande.

Passo 1: che ciclo? Indefinito con sentinella: `while`, leggendo prima del ciclo e alla fine del corpo.

Passo 2: il massimo. Non lo si può inizializzare a 0 (se i numeri sono tutti negativi sarebbe sbagliato): si usa il primo valore letto.

```
// file: conta.cpp
#include <iostream>
using namespace std;

int main() {
    int x, pos = 0, neg = 0, massimo = 0;
    bool primo = true;
    cin >> x;
    while (x != 0) {
        if (x > 0) pos++;
        else neg++;
```

```
            if (primo || x > massimo) {
                massimo = x;
                primo = false;
            }
            cin >> x;
        }
        cout << pos << " positivi, " << neg << " negativi" << endl;
        if (!primo) cout << "massimo " << massimo << endl;
        return 0;
    }
```

Passo 3: le prove. Con `-4 -9 -2 0` stampa 0 positivi, 3 negativi, massimo -2; con `3 8 -1 8 0`, 3 positivi, 1 negativo, massimo 8; con `0` solo i conteggi a zero. Provare sempre i casi limite: nessun dato, tutti negativi, valori uguali.

Guidato 3. La somma di una serie. Calcolare $\sum_{k=1}^{n} \frac{1}{k^2}$ e confrontarla con $\pi^2/6$.

Passo 1: il ciclo. Definito, da 1 a n: `for`.

Passo 2: il tranello. `1 / (k * k)` è una divisione intera, che vale 0 per $k \geq 2$: bisogna scrivere `1.0 / (k * k)`.

```
    // file: serie.cpp
    #include <iostream>
    #include <cmath>
    using namespace std;

    int main() {
        int n;
        cin >> n;
        double s = 0;
        for (int k = 1; k <= n; k++)
            s += 1.0 / (k * k);
        double esatto = M_PI * M_PI / 6;
        cout << s << " " << esatto - s << endl;
        return 0;
    }
```

Passo 3: il risultato. Con $n = 1000$ stampa `1.64393 0.0009995`: l'errore è circa $1/n$, la serie converge lentamente. `<cmath>` dà `sqrt`, `pow`, `fabs` e la costante `M_PI`.

Riepilogo

Il calcolatore di von Neumann ha CPU (unità di controllo e ALU), memoria centrale, memorie di massa, periferiche e bus; tutto è rappresentato con bit: naturali in base 2 (e 16), interi in complemento a 2, reali in virgola mobile con approssimazioni, caratteri con il codice ASCII. Un algoritmo è una sequenza finita di passi non ambigui; il programma in C++ passa per preprocessore, compilatore e linker, e gli errori possono essere di compilazione, di esecuzione o logici. Un programma C++ comincia da `main`, legge con `cin` e scrive con `cout`; le variabili hanno nome, tipo e valore. La divisione fra interi è intera, gli operatori logici sono in cortocircuito, `=` assegna e `==` confronta. Le condizioni si scrivono con `if-else` e `switch` (con `break`), le ripetizioni con `while`, `do-while` (almeno una volta, per validare) e `for` (con contatore); `break` e `continue` modificano il flusso dei cicli.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (FI-05, FI-06)**

Rappresentazioni

con n bit: 2^n valori; complemento a 2: $[-2^{n-1}, 2^{n-1}-1]$; $-x = \bar{x} + 1$; 1 cifra esadecimale $= 4$ bit.

ASCII: `'0'` = 48, `'A'` = 65, `'a'` = 97; cifra `c - '0'`; maiuscola `c - 'a' + 'A'`.

Schemi C++

```
if (c) { ... } else if (c2) { ... } else { ... }
switch (v) { case k: ...; break; default: ...; }
while (c) { ... }               // 0 o piu' volte
do { ... } while (c);           // almeno una volta
for (int i = 0; i < n; i++) { ... }
```

Attenzione	Giusto
`7 / 2` vale 3	`7 / 2.0`, `static_cast<double>(a) / b`
`if (x = 5)`	`if (x == 5)`
`0 < x < 10`	`(x > 0) && (x < 10)`
`x++` e `++x`	vecchio valore e poi incremento; incremento e nuovo valore
`switch` senza `break`	ogni `case` finisce con `break`

Termini per l'orale

Termine	In una riga
Von Neumann	programma e dati nella stessa memoria, eseguiti dalla CPU
Bit, byte	cifra binaria; 8 bit
Complemento a 2	rappresentazione degli interi con segno
Algoritmo	sequenza finita di passi non ambigui ed eseguibili
Compilatore, linker	traduce il sorgente; unisce codice oggetto e librerie
Programmazione strutturata	solo sequenza, selezione, iterazione
Cortocircuito	il secondo operando di &&, `||` si valuta solo se serve
Ciclo definito, indefinito	con contatore; con condizione o sentinella

Domande che tornano spesso

- Componenti del calcolatore e ciclo della CPU.
- Convertire fra decimale, binario ed esadecimale; il complemento a 2.
- Fasi della compilazione e tipi di errore.
- Differenza fra `while`, `do-while` e `for`; a che cosa servono `break` e `continue`.
- Che cosa succede con la divisione fra interi; precedenza e cortocircuito.

4 · Funzioni, array, stringhe, file e strutture

In questo modulo. Si usano i tipi, le espressioni e i cicli del modulo 3. Si impara a dividere un programma in **funzioni**, con il passaggio dei parametri per valore e per riferimento, e a gestire collezioni di dati: gli **array** a una e due dimensioni, con gli algoritmi di base (somma, massimo, ricerca, ordinamento), le **stringhe** come array di caratteri, i **file** di testo in lettura e in scrittura e le **strutture**, che raggruppano dati di tipo diverso. È il materiale della prova pratica del primo semestre. Alla fine del modulo si sa scrivere il prototipo di una funzione da una descrizione a parole, come nella prova di teoria, e un programma che legge dati da un file, li mette in array di strutture e li elabora con funzioni.

> **Aggiunta in revisione · fonte: slide del corso (programmazione in C++, primo semestre), esercitazioni E04-E07; H.M. Deitel e P.J. Deitel, *C++. Fondamenti di programmazione*, Apogeo (FI-06, FI-07)**
>
> Il modulo è scritto in revisione dal materiale del corso. I programmi sono compilati con `g++ -std=c++17 -Wall -Wextra` ed eseguiti da `strumenti/verifica.py`. Come nel corso si usano gli array e le stringhe "alla C" (`char[]` e `<cstring>`), non `vector` e `string` della libreria standard.

4.1 Le funzioni

Una **funzione** è un sottoprogramma con un nome, dei **parametri** e un **valore di ritorno**. Serve a non ripetere codice, a dividere un problema in parti più semplici e a dare un nome a un'operazione. Si scrive il **prototipo** (la dichiarazione: tipo di ritorno, nome, tipi dei parametri, con il `;`) prima di `main`, e la **definizione** (con il corpo) dove si vuole:

```cpp
// file: potenza.cpp
#include <iostream>
using namespace std;

double potenza(double base, int esp);      // prototipo

int main() {
    cout << potenza(2, 10) << " " << potenza(0.5, 3) << endl;
    return 0;
}

double potenza(double base, int esp) {     // definizione
    double r = 1;
    for (int i = 0; i < esp; i++)
        r *= base;
    return r;                              // termina e restituisce r
}
```

Nella chiamata `potenza(2, 10)` i **parametri attuali** 2 e 10 vengono copiati nei **parametri formali** `base` ed `esp`. `return` termina la funzione e restituisce il valore; una funzione che non restituisce nulla ha tipo `void` e può usare `return;` senza valore.

Passaggio per valore e per riferimento. Normalmente i parametri sono passati **per valore**: la funzione lavora su una copia, e modificarla non cambia la variabile del chiamante. Con il **riferimento** `&` il parametro formale è un altro nome della variabile del chiamante, e le modifiche si vedono fuori. Serve quando una funzione deve "restituire" più valori (**parametri di uscita**) o modificare i suoi argomenti:

```cpp
// file: riferimenti.cpp
#include <iostream>
using namespace std;

void scambia(int& a, int& b) {
```

```
    int t = a;
    a = b;
    b = t;
}

void dividi(int n, int d, int& q, int& r) {   // q e r sono di uscita
    q = n / d;
    r = n % d;
}

void raddoppia(int x) { x *= 2; }             // per valore: inutile

int main() {
    int x = 3, y = 8, q, r;
    scambia(x, y);
    dividi(17, 5, q, r);
    raddoppia(x);
    cout << x << " " << y << " " << q << " " << r << endl;   // 8 3 3 2
    return 0;
}
```

Al parametro per riferimento si deve passare una variabile, non un'espressione: `scambia(x, 5)` non si compila. Per gli oggetti grandi, che non si vogliono copiare ma neanche modificare, si usa il riferimento costante `const T&`.

Visibilità. Una variabile dichiarata in un blocco (anche il corpo di una funzione) è **locale**: esiste solo lì, dal punto della dichiarazione alla graffa che chiude. Le variabili dichiarate fuori da tutte le funzioni sono **globali**, visibili ovunque: vanno evitate, perché rendono difficile capire chi modifica che cosa. Ogni chiamata ha le sue variabili locali, create all'ingresso e distrutte all'uscita.

Sovraccarico (overloading). Più funzioni possono avere lo stesso nome se differiscono per numero o tipo dei parametri: il compilatore sceglie in base agli argomenti. Non basta che differiscano per il tipo di ritorno.

```
// file: overloading.cpp
#include <iostream>
using namespace std;

int massimo(int a, int b) { return a > b ? a : b; }
double massimo(double a, double b) { return a > b ? a : b; }
int massimo(int a, int b, int c) { return massimo(massimo(a, b), c); }

int main() {
    cout << massimo(3, 9) << " " << massimo(2.5, 1.5) << " "
         << massimo(4, 11, 7) << endl;          // 9 2.5 11
    return 0;
}
```

Fra le funzioni di libreria più usate: `<cmath>` (`sqrt`, `pow`, `fabs`, `sin`…) e `<cstdlib>` (`abs`, `rand`, `srand`). `rand()` dà un intero pseudocasuale fra 0 e `RAND_MAX`; `rand() % n` uno fra 0 e $n-1$; `srand(seme)` sceglie il punto di partenza della sequenza.

4.2 Gli array

Un **array** è una sequenza di elementi dello stesso tipo, in posizioni consecutive di memoria, a cui si accede con un **indice** intero che va da 0 a $n-1$:

```
int v[5];                       // 5 interi, v[0] ... v[4], valori indefiniti
int w[5] = {3, 1, 4, 1, 5};     // inizializzati
double z[100] = {0};            // tutti a 0
const int N = 10;  int a[N];    // la dimensione deve essere una costante
```

La dimensione è fissata al momento della compilazione. Il C++ **non controlla** gli indici: `v[5]` o `v[-1]` sono errori di esecuzione che possono scrivere su altre variabili senza alcun messaggio. Un array non si può assegnare in blocco (`v = w` non si compila) né confrontare con `==`: si lavora elemento per elemento con un ciclo. Spesso si dichiara un array "abbastanza grande" e si tiene a parte il numero di elementi effettivamente usati.

Array e funzioni. Un array si passa a una funzione scrivendo il parametro come `int a[]`, insieme alla sua dimensione, perché la funzione non ha modo di conoscerla. L'array **non viene copiato**: la funzione lavora sull'originale (si vedrà nel modulo 5 che viene passato l'indirizzo del primo elemento). Se la funzione non deve modificarlo, si scrive `const int a[]`.

```cpp
// file: array.cpp
#include <iostream>
using namespace std;

const int MAX = 100;

int leggi(int a[]) {                    // restituisce quanti ne ha letti
    int n;
    do cin >> n; while (n < 1 || n > MAX);
    for (int i = 0; i < n; i++) cin >> a[i];
    return n;
}

int posMassimo(const int a[], int n) {
    int p = 0;
    for (int i = 1; i < n; i++)
        if (a[i] > a[p]) p = i;
    return p;
}

int cerca(const int a[], int n, int x) {      // ricerca lineare
    for (int i = 0; i < n; i++)
        if (a[i] == x) return i;
    return -1;                                // non trovato
}

void ordina(int a[], int n) {                 // selection sort
    for (int i = 0; i < n - 1; i++) {
        int m = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[m]) m = j;
        int t = a[i]; a[i] = a[m]; a[m] = t;
    }
}

int cercaBinaria(const int a[], int n, int x) {   // a ordinato
    int sx = 0, dx = n - 1;
    while (sx <= dx) {
        int c = (sx + dx) / 2;
        if (a[c] == x) return c;
        if (a[c] < x) sx = c + 1;
        else dx = c - 1;
    }
    return -1;
}

int main() {
    int a[MAX];
```

```
    int n = leggi(a);
    cout << "massimo in posizione " << posMassimo(a, n) << endl;
    cout << "il 7 e' in posizione " << cerca(a, n, 7) << endl;
    ordina(a, n);
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl << "dopo l'ordinamento il 7 e' in posizione "
         << cercaBinaria(a, n, 7) << endl;
    return 0;
}
```

Gli algoritmi da sapere:

- **Massimo**: si parte dal primo elemento e lo si confronta con tutti gli altri ($n - 1$ confronti).
- **Ricerca lineare**: si scorre finché si trova l'elemento; nel caso peggiore n confronti.
- **Ordinamento per selezione** (*selection sort*): al passo i si cerca il minimo fra le posizioni $i, \ldots, n-1$ e lo si scambia con l'elemento in posizione i; dopo il passo i i primi $i+1$ elementi sono ordinati e definitivi. Circa $n^2/2$ confronti.
- **Ordinamento a bolle** (*bubble sort*): si confrontano le coppie adiacenti e si scambiano quelle in ordine sbagliato; dopo ogni passata il più grande "sale" in fondo. Se una passata non fa scambi, l'array è ordinato.
- **Ricerca binaria** (solo su array ordinati): si guarda l'elemento centrale e si dimezza la zona di ricerca; al massimo circa $\log_2 n$ confronti, 7 per 100 elementi.

4.3 Le matrici

Un array a due dimensioni è un array di righe: `int m[3][4];` ha 3 righe e 4 colonne, e `m[i][j]` è l'elemento di riga `i` e colonna `j`. In memoria le righe sono una dopo l'altra (memorizzazione per righe). Si scorrono con due cicli annidati. Passando una matrice a una funzione si deve indicare il **numero delle colonne**, perché il compilatore deve sapere quanto saltare per passare da una riga alla successiva: `void stampa(int m[][4], int righe)`.

```
// file: matrici.cpp
#include <iostream>
using namespace std;

const int R = 3, C = 4;

void sommeRighe(const int m[][C], int s[]) {
    for (int i = 0; i < R; i++) {
        s[i] = 0;
        for (int j = 0; j < C; j++) s[i] += m[i][j];
    }
}

void trasposta(const int m[][C], int t[][R]) {
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++) t[j][i] = m[i][j];
}

int main() {
    int m[R][C] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};
    int s[R], t[C][R];
    sommeRighe(m, s);
    trasposta(m, t);
    for (int i = 0; i < R; i++) cout << s[i] << " ";
    cout << endl;
    for (int i = 0; i < C; i++) {
        for (int j = 0; j < R; j++) cout << t[i][j] << " ";
        cout << endl;
```

```
    }
    return 0;
}
```

4.4 Le stringhe

In C++ “alla C” una **stringa** è un array di `char` che termina con il carattere nullo `'\0'`: `char s[10] = "ciao";` occupa 5 caratteri (`c`, `i`, `a`, `o`, `\0`) e ne può contenere al massimo 9. Il terminatore è ciò che permette alle funzioni di sapere dove finisce la stringa. Per leggere:

- `cin >> s;` legge una parola (si ferma al primo spazio);
- `cin.getline(s, 10);` legge una riga intera, al massimo 9 caratteri più il terminatore.

Le funzioni di `<cstring>`:

Funzione	Effetto
`strlen(s)`	lunghezza, senza il terminatore
`strcpy(d, s)`	copia `s` in `d` (`d` deve essere abbastanza grande)
`strcat(d, s)`	accoda `s` a `d`
`strcmp(a, b)`	0 se uguali, negativo se `a` viene prima in ordine alfabetico, positivo se dopo

Servono perché sulle stringhe C non funzionano `=` e `==`: `s1 == s2` confronta gli indirizzi, non il contenuto. Per elaborare una stringa carattere per carattere si scorre finché si incontra `'\0'`; `<cctype>` offre `isdigit`, `isalpha`, `toupper`, `tolower`.

```
// file: stringhe.cpp
#include <iostream>
#include <cstring>
#include <cctype>
using namespace std;

bool palindroma(const char s[]) {
    int i = 0, j = strlen(s) - 1;
    while (i < j) {
        if (tolower(s[i]) != tolower(s[j])) return false;
        i++; j--;
    }
    return true;
}

int contaVocali(const char s[]) {
    int n = 0;
    for (int i = 0; s[i] != '\0'; i++)
        if (strchr("aeiouAEIOU", s[i]) != nullptr) n++;
    return n;
}

int main() {
    char a[40], b[40], c[80];
    cin.getline(a, 40);
    cin >> b;
    cout << strlen(a) << " " << contaVocali(a) << " "
         << palindroma(b) << endl;
    strcpy(c, a);
    strcat(c, "-");
    strcat(c, b);
    cout << c << " " << (strcmp(a, b) < 0 ? "prima" : "dopo") << endl;
```

```
    return 0;
}
```

4.5 I file

I file di testo si usano come `cin` e `cout`, con i flussi di `<fstream>`: `ifstream` per leggere, `ofstream` per scrivere. Si apre il file, si controlla che l'apertura sia riuscita, si legge o si scrive con `>>`, `<<` e `getline`, si chiude.

```
// file: voti.cpp
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream in("voti.txt");
    if (!in) {                          // apertura fallita
        cout << "file non trovato" << endl;
        return 1;
    }
    char nome[30];
    int voto, n = 0, somma = 0;
    ofstream out("promossi.txt");
    while (in >> nome >> voto) {        // si ferma a fine file
        n++;
        somma += voto;
        if (voto >= 18) out << nome << endl;
    }
    in.close();
    out.close();
    if (n > 0) cout << n << " voti, media " << (double)somma / n << endl;
    ifstream ri("promossi.txt");        // rilettura del file scritto
    while (ri >> nome) cout << nome << " ";
    cout << endl;
    return 0;
}
```

`while (in >> nome >> voto)` è il modo sicuro di leggere "fino alla fine": l'espressione è falsa quando la lettura fallisce, perché il file è finito o il dato non è del tipo atteso. Il controllo con `in.eof()` *prima* di leggere è un errore frequente: `eof` diventa vero solo dopo un tentativo di lettura fallito, e così l'ultimo dato viene elaborato due volte. Per leggere righe intere si usa `in.getline(riga, MAX)`. Aprendo un `ofstream` il file viene creato, o svuotato se esiste; per aggiungere in fondo si apre con `ofstream out("f.txt", ios::app);`.

4.6 Le strutture

Una **struttura** raggruppa sotto un nome dati di tipo diverso, i suoi **campi**:

```
// file: studenti.cpp
#include <iostream>
#include <cstring>
using namespace std;

struct Data { int g, m, a; };

struct Studente {
    char nome[30];
    Data nascita;               // struttura annidata
    double media;
};
```

```cpp
void stampa(const Studente& s) {
    cout << s.nome << " (" << s.nascita.g << "/" << s.nascita.m << "/"
         << s.nascita.a << ") " << s.media << endl;
}

void ordinaPerMedia(Studente v[], int n) {      // bubble sort, decrescente
    bool scambi = true;
    for (int k = 0; k < n - 1 && scambi; k++) {
        scambi = false;
        for (int i = 0; i < n - 1 - k; i++)
            if (v[i].media < v[i + 1].media) {
                Studente t = v[i];              // le strutture si copiano
                v[i] = v[i + 1];
                v[i + 1] = t;
                scambi = true;
            }
    }
}

int main() {
    Studente v[3] = {{"Anna", {3, 5, 2006}, 26.5},
                     {"Bruno", {12, 1, 2007}, 28.0},
                     {"Carla", {30, 9, 2006}, 24.25}};
    ordinaPerMedia(v, 3);
    for (int i = 0; i < 3; i++) stampa(v[i]);
    strcpy(v[2].nome, "Carlotta");
    cout << v[2].nome << endl;
    return 0;
}
```

Si accede ai campi con il punto: `s.media`, `s.nascita.a`. A differenza degli array, le strutture **si possono assegnare** (`t = v[i]` copia tutti i campi, compresi gli array dentro) e passare per valore; per non copiarle inutilmente le si passa per riferimento, costante se non vanno modificate. Non si possono confrontare con `==`. Un **array di strutture** è il modo naturale di rappresentare una tabella: una struttura per riga, un campo per colonna.

Esempi ed esercizi svolti

> **Aggiunta in revisione · fonte: esercizi nuovi sul modello delle prove 2016-2020 e delle esercitazioni E04-E07; programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-06, FI-07)**

Esercizio 1. Prototipi (sul modello della prova di teoria). Scrivere il prototipo di: (1) `f1`, che riceve per valore un carattere e due reali e restituisce un intero; (2) `f2`, che riceve una matrice di reali di 50 righe e 20 colonne, una stringa e, per riferimento, un reale, e non restituisce nulla; (3) `f3`, che riceve un array di strutture `Studente`, la sua dimensione e un intero per riferimento, e restituisce una struttura `Studente`; (4) `f4`, che riceve una stringa costante e restituisce un booleano.

```cpp
// file: prototipi.cpp
struct Studente { char nome[30]; double media; };

int f1(char c, double x, double y);
void f2(double m[][20], char s[], double& r);
Studente f3(Studente v[], int n, int& k);
bool f4(const char s[]);

int main() { return 0; }
```

Nella matrice il numero di righe si può omettere (o scrivere: `m[50][20]` è equivalente), quello delle colonne no. I nomi dei parametri sono facoltativi nel prototipo, ma aiutano a capire.

Esercizio 2. Da esadecimale a decimale (sul modello dell'esercizio "converti"). La funzione `valore` riceve una stringa di cifre esadecimali (0-9, A-F) e restituisce il numero; il programma rifiuta le stringhe con caratteri non validi e le richiede.

```cpp
// file: esadecimale.cpp
#include <iostream>
using namespace std;

int cifra(char c) {                 // -1 se non e' una cifra esadecimale
    if (c >= '0' && c <= '9') return c - '0';
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;
    return -1;
}

int valore(const char s[]) {
    int v = 0;
    for (int i = 0; s[i] != '\0'; i++)
        v = v * 16 + cifra(s[i]);   // schema di Horner
    return v;
}

bool valida(const char s[]) {
    if (s[0] == '\0') return false;
    for (int i = 0; s[i] != '\0'; i++)
        if (cifra(s[i]) < 0) return false;
    return true;
}

int main() {
    char s[9];
    cin >> s;
    while (!valida(s)) {
        cout << "stringa non valida, riprova" << endl;
        cin >> s;
    }
    cout << valore(s) << endl;
    return 0;
}
```

Con `B6` stampa 182; con `1G` chiede di nuovo. Lo schema di Horner evita le potenze: $\texttt{B6} = (0 \cdot 16 + 11) \cdot 16 + 6$.

Esercizio 3. Un file di misure (sul modello degli esercizi su file). Il file `misure.txt` contiene per ogni riga il nome di un sensore e tre misure intere. Scrivere un programma che, per ogni riga, stampi il nome e la media, e alla fine il sensore con la media più alta. Il numero di righe non è noto, ma è al più 50.

```cpp
// file: misure.cpp
#include <iostream>
#include <fstream>
using namespace std;

struct Sensore { char nome[20]; int m[3]; double media; };

int leggiFile(const char nomeFile[], Sensore v[], int max) {
    ifstream in(nomeFile);
    int n = 0;
    while (n < max && in >> v[n].nome >> v[n].m[0] >> v[n].m[1] >> v[n].m[2]) {
```

```
        v[n].media = (v[n].m[0] + v[n].m[1] + v[n].m[2]) / 3.0;
        n++;
    }
    return n;
}

int main() {
    Sensore v[50];
    int n = leggiFile("misure.txt", v, 50);
    int best = 0;
    for (int i = 0; i < n; i++) {
        cout << v[i].nome << " " << v[i].media << endl;
        if (v[i].media > v[best].media) best = i;
    }
    if (n > 0) cout << "migliore: " << v[best].nome << endl;
    return 0;
}
```

Il limite `n < max` impedisce di scrivere oltre la fine dell'array se il file avesse più righe del previsto; va messo **prima** della lettura nella condizione, così il cortocircuito evita di leggere una riga in più.

Esercizio 4. Matrice e diagonali. Data una matrice quadrata 5×5 di interi, scrivere una funzione che restituisca, per riferimento, la somma della diagonale principale e di quella secondaria, e una che dica se la matrice è simmetrica.

```
// file: diagonali.cpp
#include <iostream>
using namespace std;

const int N = 5;

void diagonali(const int m[][N], int& princ, int& sec) {
    princ = sec = 0;
    for (int i = 0; i < N; i++) {
        princ += m[i][i];
        sec += m[i][N - 1 - i];
    }
}

bool simmetrica(const int m[][N]) {
    for (int i = 0; i < N; i++)
        for (int j = i + 1; j < N; j++)     // solo sopra la diagonale
            if (m[i][j] != m[j][i]) return false;
    return true;
}

int main() {
    int m[N][N];
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) m[i][j] = (i + 1) * (j + 1);
    int p, s;
    diagonali(m, p, s);
    cout << p << " " << s << " " << simmetrica(m) << endl;
    m[0][4] = 0;
    cout << simmetrica(m) << endl;
    return 0;
}
```

Con $m_{ij} = (i+1)(j+1)$ la diagonale principale somma $1+4+9+16+25 = 55$, la secondaria $5+8+9+8+5 = 35$, e la matrice è simmetrica: stampa `55 35 1`, poi `0`. Il secondo ciclo parte da `i + 1`: ogni coppia si controlla una volta sola.

Esercizio 5. Parole in una frase. Contare le parole di una riga (separate da uno o più spazi) e stampare la più lunga.

```cpp
// file: parole.cpp
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char riga[200], lunga[200] = "";
    cin.getline(riga, 200);
    int parole = 0, i = 0;
    while (riga[i] != '\0') {
        while (riga[i] == ' ') i++;             // salta gli spazi
        if (riga[i] == '\0') break;
        char p[200];
        int k = 0;
        while (riga[i] != ' ' && riga[i] != '\0') p[k++] = riga[i++];
        p[k] = '\0';                             // chiude la parola
        parole++;
        if (strlen(p) > strlen(lunga)) strcpy(lunga, p);
    }
    cout << parole << " " << lunga << endl;
    return 0;
}
```

Con `il compilatore segnala un errore` stampa `5 compilatore`. Dimenticare `p[k] = '\0'` è l'errore tipico: la parola continuerebbe con quello che c'è in memoria.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-06, FI-07)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. Valore o riferimento? Che cosa stampa il programma?

```cpp
// file: passaggi.cpp
#include <iostream>
using namespace std;

void f(int a, int& b, int c[]) {
    a++;
    b += a;
    c[0] = a + b;
}

int main() {
    int x = 1, y = 10, v[2] = {0, 0};
    f(x, y, v);
    f(y, x, v);
    cout << x << " " << y << " " << v[0] << endl;
```

```
    return 0;
}
```

Passo 1: prima chiamata. `a` è una copia di `x` (1) e diventa 2; `b` è `y`, che diventa 12; `v[0] = 2 + 12 = 14`. Dopo: `x = 1`, `y = 12`.

Passo 2: seconda chiamata. `a` è una copia di `y` (12) e diventa 13; `b` è `x`, che diventa $1 + 13 = 14$; `v[0] = 13 + 14 = 27`.

Passo 3: la stampa. `14 12 27`. L'array è modificato anche senza `&`, perché non viene mai copiato.

Guidato 2. Ordinare e fondere. Due array ordinati `a` (n elementi) e `b` (m elementi) vanno fusi in un terzo array ordinato.

Passo 1: l'idea. Due indici `i` e `j`; a ogni passo si copia in `c` il più piccolo fra `a[i]` e `b[j]` e si avanza quell'indice.

Passo 2: la fine. Quando uno dei due array finisce, si copia il resto dell'altro.

```
// file: fusione.cpp
#include <iostream>
using namespace std;

int fondi(const int a[], int n, const int b[], int m, int c[]) {
    int i = 0, j = 0, k = 0;
    while (i < n && j < m)
        if (a[i] <= b[j]) c[k++] = a[i++];
        else c[k++] = b[j++];
    while (i < n) c[k++] = a[i++];
    while (j < m) c[k++] = b[j++];
    return k;
}

int main() {
    int a[] = {1, 4, 9, 12}, b[] = {2, 3, 10}, c[7];
    int k = fondi(a, 4, b, 3, c);
    for (int i = 0; i < k; i++) cout << c[i] << " ";
    cout << endl;
    return 0;
}
```

Passo 3: il risultato. `1 2 3 4 9 10 12`, con al più $n + m$ confronti: è il cuore dell'ordinamento *merge sort*.

Guidato 3. Statistiche su un file di temperature. Il file `temperature.txt` contiene una temperatura reale per riga; contare i giorni sopra la media.

Passo 1: due passate. La media si conosce solo alla fine: o si rilegge il file, o si mettono i valori in un array. Si sceglie l'array (al più 365 valori).

Passo 2: il programma.

```
// file: temperature.cpp
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream in("temperature.txt");
    double t[365], somma = 0;
    int n = 0;
    while (n < 365 && in >> t[n]) somma += t[n++];
    double media = somma / n;
    int sopra = 0;
```

```
    for (int i = 0; i < n; i++)
        if (t[i] > media) sopra++;
    cout << n << " " << media << " " << sopra << endl;
    return 0;
}
```

Passo 3: la prova. Con i valori 12.5, 14, 9.5, 16, 13 stampa `5 13 2`: media 13, sopra la media 14 e 16. Con un file vuoto `n` sarebbe 0 e la divisione darebbe un risultato non numerico: in un programma d'esame va controllato.

Riepilogo

Una funzione ha prototipo e definizione; i parametri passano per valore (copia) o per riferimento (&, le modifiche si vedono fuori, per i parametri di uscita); le variabili locali vivono solo nel loro blocco; più funzioni possono avere lo stesso nome con parametri diversi. Gli array hanno dimensione costante, indici da 0 a $n-1$ non controllati, non si assegnano né si confrontano in blocco e si passano alle funzioni senza copia, con la dimensione a parte; gli algoritmi base sono massimo, ricerca lineare e binaria, ordinamento per selezione e a bolle, fusione. Le matrici sono array di righe e nei parametri richiedono il numero di colonne. Le stringhe sono array di `char` terminati da `'\0'` e si gestiscono con `<cstring>`. I file di testo si leggono con `ifstream` controllando la lettura nella condizione del ciclo e si scrivono con `ofstream`. Le strutture raggruppano campi di tipi diversi, si assegnano e si passano come un tutto, e negli array rappresentano tabelle.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (FI-06, FI-07)**

Schemi

```
tipo nome(tipo p1, tipo& p2, const tipo a[], int n);   // prototipo
void f(int m[][COL], int righe);                       // matrice
char s[N];  cin.getline(s, N);  strlen strcpy strcat strcmp
ifstream in("f.txt");  if (!in) ...;  while (in >> x) ...;  in.close();
ofstream out("f.txt");  out << x << endl;  // ios::app per accodare
struct T { tipo campo; ... };  T v[N];  v[i].campo
```

Algoritmo	Idea	Confronti
ricerca lineare	scorre tutto	fino a n
ricerca binaria	dimezza (array ordinato)	circa $\log_2 n$
selection sort	minimo del resto in posizione i	circa $n^2/2$
bubble sort	scambi fra adiacenti, il massimo sale	fino a circa $n^2/2$
fusione	il più piccolo dei due in testa	fino a $n+m$

Termini per l'orale

Termine	In una riga
Prototipo	dichiarazione della funzione: tipo, nome, parametri
Parametro formale, attuale	quello della definizione; quello della chiamata
Per valore, per riferimento	copia; alias della variabile del chiamante
Visibilità (scope)	parte del programma in cui un nome esiste
Overloading	stesso nome, parametri diversi
Terminatore `'\0'`	segna la fine di una stringa C
Flusso (stream)	sequenza di caratteri da o verso tastiera, schermo, file
Struttura	tipo composto da campi di tipi diversi

Domande che tornano spesso

- Differenza fra passaggio per valore e per riferimento; perché un array si comporta come per riferimento?
- Perché nel parametro matrice serve il numero di colonne?
- Come si legge un file fino alla fine senza usare `eof()` prima della lettura?
- Che cosa succede con un indice fuori dall'array?
- Descrivere un algoritmo di ordinamento e la ricerca binaria.

5 · Puntatori, memoria dinamica, ricorsione e template

In questo modulo. Si usano le funzioni, gli array e le strutture del modulo 4. È la parte di "C++ avanzato" con cui comincia il secondo semestre. Si vede che cos'è un indirizzo di memoria e come lo si manipola con i **puntatori**, il legame fra puntatori e array, l'aritmetica dei puntatori, e la differenza fra puntatori e riferimenti. Poi l'**allocazione dinamica**: creare variabili, array e matrici di dimensione decisa durante l'esecuzione, e liberarli. Poi la **ricorsione**, cioè le funzioni che chiamano sé stesse, e infine i **template di funzione**, che scrivono una funzione una volta sola per tutti i tipi. Alla fine del modulo si sa seguire a mano un programma con puntatori, gestire array dinamici senza perdite di memoria, scrivere e tracciare funzioni ricorsive e scrivere template di funzione.

> **Aggiunta in revisione · fonte: slide del corso (Fondamenti, secondo semestre: 3. Programmazione avanzata in C++), esercitazioni del 25 febbraio, 4 e 11 marzo; H.M. Deitel e P.J. Deitel, *C++. Fondamenti di programmazione*, Apogeo (FI-08)**
>
> Il modulo è scritto in revisione dal materiale del corso. I programmi sono compilati con `g++ -std=c++17 -Wall -Wextra` ed eseguiti da `strumenti/verifica.py`. Gli indirizzi stampati da un programma cambiano da un'esecuzione all'altra: negli esempi si stampano solo valori.

5.1 Indirizzi e puntatori

Ogni variabile occupa una zona di memoria, che ha un **indirizzo**. L'operatore & applicato a una variabile ne dà l'indirizzo. Un **puntatore** è una variabile che contiene un indirizzo; il suo tipo dice a che tipo di dato punta: `int* p;` è un puntatore a `int`. L'operatore * (**dereferenziazione**) applicato a un puntatore dà la variabile puntata.

```
// file: puntatori.cpp
#include <iostream>
using namespace std;

int main() {
    int x = 5, y = 9;
    int* p = &x;            // p punta a x
    *p = 7;                 // modifica x attraverso p
    p = &y;                 // ora p punta a y
    *p += *p;               // y = y + y
    int* q = p;             // q e p puntano alla stessa variabile
    *q = *q - 1;
    cout << x << " " << y << " " << (p == q) << endl;   // 7 17 1
    int* r = nullptr;       // puntatore che non punta a nulla
    if (r == nullptr) cout << "r e' nullo" << endl;
    return 0;
}
```

Il simbolo * ha due significati: nella dichiarazione (`int* p`) dice che `p` è un puntatore, in un'espressione (`*p`) dereferenzia. Attenzione: `int* a, b;` dichiara un puntatore `a` e un intero `b`. Un puntatore non inizializzato contiene un indirizzo a caso, e dereferenziarlo è un errore di esecuzione; per indicare "nessun oggetto" si usa il puntatore nullo, `nullptr` (o `NULL`, o 0 nel codice più vecchio). Dereferenziare un puntatore nullo fa terminare il programma.

Puntatori come parametri. Passare a una funzione l'indirizzo di una variabile le permette di modificarla, come il riferimento del modulo 4 ma in modo esplicito:

```
void scambia(int* a, int* b) { int t = *a; *a = *b; *b = t; }
scambia(&x, &y);                  // si passano gli indirizzi
```

Puntatori e riferimenti. Un riferimento è un altro nome di una variabile: va inizializzato quando si dichiara, non si può "spostare" su un'altra variabile e non può essere nullo; si usa senza *. Un puntatore è una variabile a

sé, può cambiare oggetto, può essere nullo, e va dereferenziato. Per i parametri di uscita si preferisce il riferimento; i puntatori servono per la memoria dinamica e le strutture collegate.

Puntatori e `const`. `const int* p` è un puntatore a un intero costante: non si può modificare `*p`, ma `p` può puntare altrove. `int* const p` è un puntatore costante: punta sempre alla stessa variabile, che però si può modificare. Si legge da destra a sinistra.

5.2 Puntatori e array

Il nome di un array, in quasi tutte le espressioni, vale l'indirizzo del suo primo elemento: `a` equivale a `&a[0]`. Per questo un array passato a una funzione non viene copiato: si passa questo indirizzo, e il parametro `int a[]` è in realtà un `int* a`.

Aritmetica dei puntatori. Se `p` punta a un elemento di un array, `p + i` punta all'elemento i posizioni dopo: il compilatore moltiplica i per la dimensione del tipo. Quindi

$$\texttt{a[i]} \equiv \texttt{*(a + i)}, \qquad \texttt{\&a[i]} \equiv \texttt{a + i}.$$

Si possono incrementare i puntatori (`p++` passa all'elemento successivo), confrontarli e sottrarli: se `p` e `q` puntano allo stesso array, `q - p` è il numero di elementi fra loro. Non ha senso sommare due puntatori.

```cpp
// file: aritmetica.cpp
#include <iostream>
using namespace std;

int lunghezza(const char* s) {        // come strlen
    const char* p = s;
    while (*p != '\0') p++;
    return p - s;                     // differenza di puntatori
}

int somma(const int* inizio, const int* fine) {   // [inizio, fine)
    int s = 0;
    for (const int* p = inizio; p != fine; p++) s += *p;
    return s;
}

int main() {
    int a[6] = {4, 8, 15, 16, 23, 42};
    int* p = a + 2;                   // punta a a[2]
    cout << *p << " " << *(p + 1) << " " << p[2] << " " << *a + 1 << endl;
    cout << somma(a, a + 6) << " " << somma(a + 1, a + 3) << endl;
    cout << lunghezza("puntatore") << endl;
    return 0;
}
```

Stampa `15 16 23 5` (attenzione a `*a + 1`, che è `a[0] + 1`, non `a[1]`), poi `108 23` e `9`. Anche `p[2]` è lecito: l'indicizzazione funziona su qualunque puntatore.

5.3 La memoria dinamica

Le variabili locali stanno nello **stack** (la pila delle chiamate): vengono create all'ingresso nel blocco e distrutte all'uscita, e la loro dimensione deve essere nota quando si compila. Per creare oggetti la cui dimensione o durata si decide durante l'esecuzione si usa lo **heap** (la memoria libera), con l'operatore **`new`**, che alloca e restituisce l'indirizzo, e **`delete`**, che libera:

```cpp
int* p = new int(5);        // un intero, inizializzato a 5
delete p;                   // liberato
int* v = new int[n];        // array di n interi, n letto a run time
delete[] v;                 // per gli array: delete[]
```

Un oggetto allocato con `new` vive finché non viene liberato con `delete`, indipendentemente dalla funzione che l'ha creato: per questo una funzione può allocare un array e restituirne l'indirizzo. Gli errori da evitare:

- **perdita di memoria** (*memory leak*): perdere l'ultimo puntatore a un oggetto allocato senza liberarlo; la memoria resta occupata fino alla fine del programma;
- **puntatore pendente** (*dangling pointer*): usare un puntatore dopo il `delete`, o restituire l'indirizzo di una variabile locale, che non esiste più; conviene mettere il puntatore a `nullptr` dopo il `delete`;
- **doppio `delete`**, o `delete` di un indirizzo non ottenuto con `new`;
- usare `delete` per un array (serve `delete[]`).

```cpp
// file: dinamico.cpp
#include <iostream>
using namespace std;

int* crea(int n, int valore) {
    int* v = new int[n];
    for (int i = 0; i < n; i++) v[i] = valore + i;
    return v;                          // l'array sopravvive alla funzione
}

int* allunga(int* v, int n, int nuovo) {   // array di n + 1 elementi
    int* w = new int[n + 1];
    for (int i = 0; i < n; i++) w[i] = v[i];
    w[n] = nuovo;
    delete[] v;                        // il vecchio non serve piu'
    return w;
}

int main() {
    int n;
    cin >> n;
    int* v = crea(n, 10);
    v = allunga(v, n, 99);
    n++;
    for (int i = 0; i < n; i++) cout << v[i] << " ";
    cout << endl;
    delete[] v;
    v = nullptr;
    return 0;
}
```

Matrici dinamiche. Una matrice di r righe e c colonne decise a run time si costruisce come array di r puntatori, ciascuno a un array di c elementi; si libera in ordine inverso.

```cpp
// file: matrice_dinamica.cpp
#include <iostream>
using namespace std;

int** creaMatrice(int r, int c) {
    int** m = new int*[r];
    for (int i = 0; i < r; i++) m[i] = new int[c];
    return m;
}

void distruggi(int** m, int r) {
    for (int i = 0; i < r; i++) delete[] m[i];   // prima le righe
    delete[] m;                                  // poi l'array di puntatori
}
```

```
int main() {
    int r, c;
    cin >> r >> c;
    int** m = creaMatrice(r, c);
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) m[i][j] = i * c + j;
    cout << m[r - 1][c - 1] << " " << m[1][0] << endl;
    distruggi(m, r);
    return 0;
}
```

Puntatori a strutture. Se `p` punta a una struttura, si accede ai campi con `(*p).campo` o, più comodo, con **`p->campo`**. È l'operatore che si userà per gli oggetti allocati dinamicamente (modulo 6) e per i nodi delle liste (modulo 7).

5.4 La ricorsione

Una funzione è **ricorsiva** se chiama sé stessa. Funziona se ci sono un **caso base**, risolto direttamente, e un **passo ricorsivo** che riconduce il problema a uno più piccolo, avvicinandosi al caso base. Il modello è l'induzione del corso di Analisi: $n! = n \cdot (n-1)!$ con $0! = 1$.

```
// file: ricorsione.cpp
#include <iostream>
using namespace std;

long fattoriale(int n) {
    if (n == 0) return 1;                // caso base
    return n * fattoriale(n - 1);        // passo ricorsivo
}

int mcd(int a, int b) { return b == 0 ? a : mcd(b, a % b); }

int sommaArray(const int a[], int n) {    // somma dei primi n
    if (n == 0) return 0;
    return sommaArray(a, n - 1) + a[n - 1];
}

bool palindroma(const char s[], int i, int j) {
    if (i >= j) return true;
    return s[i] == s[j] && palindroma(s, i + 1, j - 1);
}

void binario(int n) {                    // stampa i bit, dal piu' alto
    if (n >= 2) binario(n / 2);
    cout << n % 2;
}

int mosse = 0;
void hanoi(int n, char da, char a, char via) {
    if (n == 0) return;
    hanoi(n - 1, da, via, a);            // n-1 dischi sul piolo d'appoggio
    mosse++;                             // il disco grande va a destinazione
    hanoi(n - 1, via, a, da);            // gli n-1 sopra di lui
}

int main() {
    int v[] = {3, 1, 4, 1, 5};
    cout << fattoriale(10) << " " << mcd(84, 36) << " "
```

```
         << sommaArray(v, 5) << " " << palindroma("radar", 0, 4) << endl;
    binario(45);
    cout << endl;
    hanoi(5, 'A', 'C', 'B');
    cout << mosse << endl;
    return 0;
}
```

Stampa `3628800 12 14 1`, `101101` e 31. Ogni chiamata ha il suo **record di attivazione** sullo stack, con i parametri e le variabili locali: `fattoriale(3)` aspetta `fattoriale(2)`, che aspetta `fattoriale(1)`, fino a `fattoriale(0)` che restituisce 1; poi i risultati "risalgono" moltiplicandosi. Senza caso base (o se il passo non si avvicina al caso base) le chiamate non finiscono e lo stack si esaurisce (*stack overflow*).

Nella funzione `binario` la stampa viene **dopo** la chiamata ricorsiva: così i bit escono dal più significativo, al contrario dell'ordine in cui si calcolano. Le **torri di Hanoi** mostrano la forza della ricorsione: spostare n dischi richiede di spostarne $n-1$, il grande, e di nuovo $n-1$, quindi $M(n) = 2M(n-1) + 1$ con $M(0) = 0$, cioè $M(n) = 2^n - 1$.

Ricorsione o iterazione? Ogni funzione ricorsiva si può scrivere con un ciclo, e viceversa. La versione ricorsiva è spesso più chiara quando il problema ha una struttura ricorsiva (Hanoi, alberi, divide et impera), ma costa una chiamata per livello, e alcune ricorsioni ripetono molto lavoro: Fibonacci con $F(n) = F(n-1) + F(n-2)$ ricalcola gli stessi valori un numero esponenziale di volte, mentre un ciclo con due variabili impiega n passi.

5.5 I template di funzione

Molte funzioni fanno la stessa cosa su tipi diversi: scambiare due valori, trovare il massimo di un array, ordinare. Invece di scrivere una versione per tipo (overloading), si scrive un **template**, con un parametro di tipo:

```
// file: template.cpp
#include <iostream>
using namespace std;

template <typename T>
void scambia(T& a, T& b) {
    T t = a;
    a = b;
    b = t;
}

template <typename T>
T massimo(const T a[], int n) {
    T m = a[0];
    for (int i = 1; i < n; i++)
        if (m < a[i]) m = a[i];          // T deve avere l'operatore <
    return m;
}

template <typename T>
void ordina(T a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (a[j + 1] < a[j]) scambia(a[j], a[j + 1]);
}

int main() {
    int x = 1, y = 2;
    scambia(x, y);
    double d[] = {2.5, -1, 9.75, 3};
    char c[] = {'t', 'e', 'm', 'p'};
```

```
    ordina(c, 4);
    cout << x << y << " " << massimo(d, 4) << " " << massimo(c, 4) << " "
         << c[0] << c[1] << c[2] << c[3] << endl;
    return 0;
}
```

Il compilatore **istanzia** il template, cioè genera la funzione per il tipo usato, deducendolo dagli argomenti: `scambia(x, y)` crea `scambia<int>`. Il tipo si può anche indicare esplicitamente, `massimo<double>(d, 4)`. Il codice del template deve avere senso per ogni tipo con cui lo si usa: `massimo` richiede che `T` abbia l'operatore `<` (lo si definirà per le classi nel modulo 6). Si può scrivere `class` al posto di `typename`, con lo stesso significato.

Esempi ed esercizi svolti

> **Aggiunta in revisione · fonte: esercizi nuovi sul modello delle prove 2016-2020 e delle esercitazioni del secondo semestre; programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-08)**

Esercizio 1. Trovare e correggere gli errori (sul modello della prova di teoria).

```
1   int* crea(int n) {
2       int v[n];
3       for (int i = 0; i <= n; i++) v[i] = 0;
4       return v;
5   }
6   int main() {
7       int* p;
8       *p = 3;
9       int* a = crea(10);
10      delete a;
11  }
```

Riga 2: la dimensione di un array locale deve essere costante, e comunque un array locale sparisce all'uscita: serve `int* v = new int[n];`. Riga 3: l'ultimo indice valido è `n - 1`, la condizione è `i < n`. Riga 4: restituire l'indirizzo di una variabile locale crea un puntatore pendente (corretto dalla riga 2 con `new`). Riga 8: `p` non è inizializzato; serve per esempio `int x; int* p = &x;`. Riga 10: un array si libera con `delete[] a;`.

Esercizio 2. Che cosa stampa?

```
// file: traccia_puntatori.cpp
#include <iostream>
using namespace std;

void f(int* p, int& r, int v) {
    *p += v;
    r = *p * 2;
    p = &v;          // cambia solo la copia locale del puntatore
    *p = 100;
}

int main() {
    int a[3] = {1, 2, 3};
    int* q = a;
    f(q + 1, a[0], 5);
    q += 2;
    *q = *(q - 1) + a[0];
    cout << a[0] << " " << a[1] << " " << a[2] << endl;
    return 0;
}
```

In `f`, `p` punta ad `a[1]`, che diventa $2 + 5 = 7$; `r` è `a[0]`, che diventa 14; poi `p` punta alla copia locale `v`, e `*p = 100` cambia solo quella. Nel `main`, `q` punta ad `a[2]`, che diventa $a[1] + a[0] = 7 + 14 = 21$. Stampa `14 7 21`.

Esercizio 3. Array dinamico dei valori sopra soglia. La funzione riceve un array, la sua dimensione e una soglia; restituisce un nuovo array, allocato dinamicamente, con i soli elementi maggiori della soglia, e la sua dimensione per riferimento.

```cpp
// file: filtra.cpp
#include <iostream>
using namespace std;

int* filtra(const int a[], int n, int soglia, int& m) {
    m = 0;
    for (int i = 0; i < n; i++)         // prima passata: quanti sono
        if (a[i] > soglia) m++;
    int* b = new int[m];
    int k = 0;
    for (int i = 0; i < n; i++)         // seconda passata: copia
        if (a[i] > soglia) b[k++] = a[i];
    return b;
}

int main() {
    int a[] = {12, 5, 30, 8, 19, 7, 25};
    int m;
    int* b = filtra(a, 7, 10, m);
    cout << m << ":";
    for (int i = 0; i < m; i++) cout << " " << b[i];
    cout << endl;
    delete[] b;
    return 0;
}
```

Stampa `4: 12 30 19 25`. Le due passate evitano di allocare “abbastanza” memoria a caso; chi chiama la funzione deve ricordarsi del `delete[]`.

Esercizio 4. Ricorsione su un array: il massimo e la ricerca binaria.

```cpp
// file: ricorsione_array.cpp
#include <iostream>
using namespace std;

int massimo(const int a[], int n) {            // n >= 1
    if (n == 1) return a[0];
    int m = massimo(a + 1, n - 1);             // massimo del resto
    return a[0] > m ? a[0] : m;
}

int cerca(const int a[], int sx, int dx, int x) {
    if (sx > dx) return -1;
    int c = (sx + dx) / 2;
    if (a[c] == x) return c;
    if (a[c] < x) return cerca(a, c + 1, dx, x);
    return cerca(a, sx, c - 1, x);
}

int main() {
    int a[] = {2, 3, 5, 7, 11, 13, 17, 19};
    cout << massimo(a, 8) << " " << cerca(a, 0, 7, 13) << " "
```

```
         << cerca(a, 0, 7, 4) << endl;
    return 0;
}
```

Stampa `19 5 -1`. In `massimo(a + 1, n - 1)` si passa il “resto” dell’array con l’aritmetica dei puntatori.

Esercizio 5. Fibonacci: ricorsivo e iterativo. Quante chiamate fa la versione ricorsiva per $F(20)$?

```
// file: fibonacci.cpp
#include <iostream>
using namespace std;

long chiamate = 0;

long fibR(int n) {
    chiamate++;
    if (n < 2) return n;
    return fibR(n - 1) + fibR(n - 2);
}

long fibI(int n) {
    long a = 0, b = 1;                  // F(0), F(1)
    for (int i = 0; i < n; i++) {
        long t = a + b;
        a = b;
        b = t;
    }
    return a;
}

int main() {
    cout << fibR(20) << " " << fibI(20) << " " << chiamate << endl;
    return 0;
}
```

Stampa `6765 6765 21891`: quasi ventiduemila chiamate contro 20 giri di ciclo. Il numero di chiamate $C(n) = C(n-1) + C(n-2) + 1$ cresce come i numeri di Fibonacci stessi.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-08)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. Tracciare una ricorsione. Che cosa stampa `f(4)`?

```
// file: traccia_ricorsione.cpp
#include <iostream>
using namespace std;

int f(int n) {
    if (n <= 1) return 1;
    int a = f(n - 1);
    int b = f(n - 2);
    int r = a + b;
    cout << n << ":" << r << " ";
    return r;
}
```

```cpp
int main() {
    cout << endl << f(4) << endl;
    return 0;
}
```

Passo 1: l'albero delle chiamate. `f(4)` chiama prima `f(3)` e poi `f(2)` (le due chiamate sono in istruzioni separate apposta: in `f(n - 1) + f(n - 2)` l'ordine di valutazione dei due operandi non è stabilito dal linguaggio); `f(3)` chiama `f(2)` e `f(1)`; ogni `f(2)` chiama `f(1)` e `f(0)`.

Passo 2: l'ordine delle stampe. Si stampa dopo le due chiamate, quindi "dal basso": prima `f(2)` dentro `f(3)` (`2:2`), poi `f(3)` (`3:3`), poi il secondo `f(2)` (`2:2`), infine `f(4)` (`4:5`).

Passo 3: l'uscita. La riga vuota iniziale, poi `2:2 3:3 2:2 4:5`, e dopo il valore restituito `5`. Attenzione: in C++17 l'operando di sinistra di `<<` si valuta prima di quello di destra, perciò `endl` è scritto prima della chiamata.

Guidato 2. Template per contare. Scrivere un template che conti quante volte un valore compare in un array, e provarlo su interi e caratteri.

Passo 1: il prototipo. `template <typename T> int conta(const T a[], int n, const T& x);`

Passo 2: il corpo. Un ciclo con `if (a[i] == x)`: il tipo deve avere `==`.

```cpp
// file: conta_template.cpp
#include <iostream>
using namespace std;

template <typename T>
int conta(const T a[], int n, const T& x) {
    int k = 0;
    for (int i = 0; i < n; i++)
        if (a[i] == x) k++;
    return k;
}

int main() {
    int v[] = {1, 3, 1, 1, 2};
    char s[] = "ricorsione";
    cout << conta(v, 5, 1) << " " << conta(s, 10, 'r') << endl;
    return 0;
}
```

Passo 3: il controllo. Stampa `3 2`. Con `conta(v, 5, 1.0)` il compilatore non saprebbe se `T` è `int` o `double` e darebbe errore: si risolve con `conta<int>(v, 5, 1)`.

Guidato 3. Una matrice triangolare dinamica. Allocare il triangolo di Tartaglia con n righe, dove la riga i ha $i + 1$ elementi.

Passo 1: la struttura. Un array di n puntatori; la riga i è un array di $i + 1$ interi: righe di lunghezza diversa, impossibile con una matrice statica.

Passo 2: i valori. Gli estremi valgono 1, gli altri sono la somma dei due sopra.

```cpp
// file: tartaglia.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    int** t = new int*[n];
    for (int i = 0; i < n; i++) {
```

```
            t[i] = new int[i + 1];
            t[i][0] = t[i][i] = 1;
            for (int j = 1; j < i; j++) t[i][j] = t[i - 1][j - 1] + t[i - 1][j];
        }
        for (int j = 0; j < n; j++) cout << t[n - 1][j] << " ";
        cout << endl;
        for (int i = 0; i < n; i++) delete[] t[i];
        delete[] t;
        return 0;
    }
```

Passo 3: la prova. Con $n = 7$ l'ultima riga è `1 6 15 20 15 6 1`, i coefficienti $\binom{6}{k}$ del modulo 2. Si liberano prima le righe, poi l'array dei puntatori.

Riepilogo

Un puntatore contiene un indirizzo: & dà l'indirizzo di una variabile, * la variabile puntata, `nullptr` indica nessun oggetto. Il nome di un array è l'indirizzo del primo elemento e `a[i]` equivale a `*(a + i)`; l'aritmetica dei puntatori conta in elementi. Un riferimento è un alias fisso; un puntatore è una variabile che può cambiare oggetto ed essere nullo. Con `new` e `delete` (o `new[]` e `delete[]`) si gestiscono oggetti nello heap, che vivono finché non vengono liberati: bisogna evitare perdite di memoria, puntatori pendenti e doppi `delete`; le matrici dinamiche sono array di puntatori a righe. Una funzione ricorsiva ha un caso base e un passo che riduce il problema; ogni chiamata ha il suo record sullo stack. I template di funzione scrivono una funzione per tutti i tipi che offrono le operazioni usate, e il compilatore li istanzia per i tipi effettivi.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (FI-08)**

Schemi

```
int* p = &x;   *p = 3;   p = nullptr;          // puntatore
a[i] == *(a + i)    &a[i] == a + i    q - p    // aritmetica
T* p = new T;  delete p;   T* v = new T[n];  delete[] v;
T** m = new T*[r];  m[i] = new T[c];  ... delete[] m[i]; delete[] m;
p->campo  ==  (*p).campo
template <typename T> T f(const T a[], int n);
```

Dichiarazione	Si può cambiare p?	Si può cambiare *p?
`int* p`	sì	sì
`const int* p`	sì	no
`int* const p`	no	sì
`const int* const p`	no	no

Hanoi: $M(n) = 2M(n-1) + 1 = 2^n - 1$; Fibonacci ricorsivo: $C(n) = C(n-1) + C(n-2) + 1$.

Termini per l'orale

Termine	In una riga
Dereferenziazione	accesso alla variabile puntata con *
Puntatore nullo	non punta a nessun oggetto
Stack, heap	memoria delle chiamate; memoria allocata con new
Memory leak	memoria allocata e non più raggiungibile
Dangling pointer	puntatore a memoria non più valida
Caso base, passo ricorsivo	soluzione diretta; riduzione a un problema più piccolo

Termine	In una riga
Record di attivazione	parametri e variabili locali di una chiamata
Istanziazione	generazione della funzione dal template per un tipo

Domande che tornano spesso

- Che cosa sono & e *? Differenza fra puntatore e riferimento.
- Perché `a[i]` equivale a `*(a + i)`? Che cosa vale `q - p`?
- Come si alloca e libera un array dinamico e una matrice dinamica? Quali errori si fanno?
- Struttura di una funzione ricorsiva; che cosa succede sullo stack; ricorsione e iterazione.
- Che cos’è un template di funzione e quando il compilatore lo istanzia?

6 · Classi e oggetti

In questo modulo. Si usano le strutture e le funzioni del modulo 4 e i puntatori, la memoria dinamica e i template del modulo 5. Si passa dalla struttura, che raggruppa solo dati, alla **classe**, che mette insieme dati e operazioni e decide che cosa è visibile da fuori. Si vedono attributi e metodi, il controllo di accesso, i costruttori (di default, con parametri, di copia) e il distruttore, i selettori e i modificatori, i metodi costanti e il puntatore `this`, il sovraccarico degli operatori, le classi che gestiscono memoria dinamica con la "regola dei tre", e i template di classe. È la base della prova pratica del secondo semestre, che chiede sempre di scrivere una classe con costruttori, selettori, modificatori, operatore di assegnamento e alcuni metodi di calcolo. Alla fine del modulo si sa scrivere una classe completa da una specifica a parole e usarla in un programma con oggetti allocati dinamicamente.

> **Aggiunta in revisione · fonte: slide del corso (Fondamenti, secondo semestre: classi, template delle classi), esercitazioni del 18 e 25 marzo, del 1° aprile, `Esercitazioni_15_classi_allocazione_dinamica`; prove 2016-2020 (FI-09)**
>
> Il modulo è scritto in revisione dal materiale del corso. Si seguono le convenzioni delle slide: attributi privati con il trattino basso (`_dim`), selettori `get…`, modificatori `set…`, operatore di assegnamento che restituisce un riferimento costante. I programmi sono compilati ed eseguiti da `strumenti/verifica.py`, con AddressSanitizer per le perdite di memoria.

6.1 Dalla struttura alla classe

Una struttura del modulo 4 è un contenitore di dati: chiunque può leggerne e modificarne i campi, anche mettendoci valori senza senso (una data con il mese 13, una frazione con denominatore 0). Nella **programmazione orientata agli oggetti** un **oggetto** ha uno **stato** (i suoi dati, gli **attributi**) e un **comportamento** (le operazioni che sa fare, i **metodi**), e una **classe** è il tipo degli oggetti, il loro "stampo".

Il **controllo di accesso** separa ciò che si usa da fuori da ciò che riguarda solo la classe. I membri **`private`** sono accessibili solo dai metodi della classe; quelli **`public`** da chiunque. Di regola gli attributi sono privati e si raggiungono solo attraverso i metodi pubblici: è l'**incapsulamento**, che permette alla classe di garantire che il suo stato sia sempre valido e di cambiare la rappresentazione interna senza toccare il codice che la usa (modulo 7). In una `class` i membri sono privati se non si dice altro, in una `struct` pubblici: è l'unica differenza.

```
// file: data.cpp
#include <iostream>
using namespace std;

class Data {
    int _g, _m, _a;                       // privati
    static int giorni(int m, int a);      // metodo d'aiuto, privato
public:
    Data();                               // costruttore di default
    Data(int g, int m, int a);            // costruttore con parametri
    int getGiorno() const { return _g; }  // selettori
    int getMese() const { return _m; }
    int getAnno() const { return _a; }
    bool setData(int g, int m, int a);    // modificatore con controllo
    void avanza();                        // giorno successivo
    void stampa() const;
};

int Data::giorni(int m, int a) {
    if (m == 2) return (a % 4 == 0 && a % 100 != 0) || a % 400 == 0 ? 29 : 28;
    if (m == 4 || m == 6 || m == 9 || m == 11) return 30;
    return 31;
```

```
}

Data::Data() : _g(1), _m(1), _a(2000) {}

Data::Data(int g, int m, int a) : _g(1), _m(1), _a(2000) {
    setData(g, m, a);                       // se non valida resta 1/1/2000
}

bool Data::setData(int g, int m, int a) {
    if (m < 1 || m > 12 || g < 1 || g > giorni(m, a)) return false;
    _g = g; _m = m; _a = a;
    return true;
}

void Data::avanza() {
    if (++_g > giorni(_m, _a)) {
        _g = 1;
        if (++_m > 12) { _m = 1; _a++; }
    }
}

void Data::stampa() const { cout << _g << "/" << _m << "/" << _a << endl; }

int main() {
    Data d(28, 2, 2028), e(31, 4, 2026);   // e non valida
    d.avanza();
    d.stampa();                             // 29/2/2028: bisestile
    d.avanza();
    d.stampa();
    e.stampa();
    cout << e.setData(31, 12, 2026) << " ";
    e.avanza();
    e.stampa();
    return 0;
}
```

La dichiarazione della classe è la sua **interfaccia**: dice che cosa si può fare con gli oggetti. Le definizioni dei metodi, fuori dalla classe, si scrivono con il nome della classe e l'operatore di **risoluzione di visibilità** `::` (`void Data::avanza()`); i metodi molto brevi si possono definire direttamente dentro la classe. Nei progetti veri l'interfaccia sta in un file di intestazione (`Data.h`) e l'implementazione in `Data.cpp`. Un metodo si chiama su un oggetto con il punto, `d.avanza()`, e lavora sugli attributi di quell'oggetto. Un metodo `static` appartiene alla classe e non a un oggetto: `giorni` non usa gli attributi.

6.2 Costruttori e distruttore

Un **costruttore** è un metodo con lo stesso nome della classe e senza tipo di ritorno, chiamato automaticamente quando un oggetto viene creato: serve a dare agli attributi valori iniziali validi. Ce ne possono essere più d'uno (overloading):

- il **costruttore di default**, senza parametri, usato in `Data d;` e quando si crea un array di oggetti; se una classe non dichiara nessun costruttore il compilatore ne genera uno che non inizializza gli attributi di tipo fondamentale;
- i **costruttori con parametri**, `Data e(31, 4, 2026);`;
- il **costruttore di copia**, `Data(const Data& d)`, usato quando si crea un oggetto come copia di un altro: `Data f(d);`, `Data f = d;`, e ogni volta che un oggetto si passa o si restituisce **per valore**. Il parametro deve essere un riferimento: se fosse per valore, per passarlo servirebbe già il costruttore di copia.

La **lista di inizializzazione** (`: _g(1), _m(1), _a(2000)`) inizializza gli attributi prima del corpo del costruttore; è il modo consigliato, e l'unico per gli attributi costanti e i riferimenti.

Il **distruttore** `~Data()` è chiamato automaticamente quando l'oggetto finisce di esistere: all'uscita dal blocco per un oggetto locale, con `delete` per uno dinamico. Non ha parametri né tipo di ritorno, e ce n'è uno solo. Serve soprattutto a liberare le risorse che l'oggetto possiede, come la memoria dinamica (6.5).

Oggetti dinamici. `Data* p = new Data(1, 3, 2027);` alloca un oggetto e chiama il costruttore; si usano i metodi con la freccia, `p->avanza()`; `delete p;` chiama il distruttore e libera. `Data* v = new Data[10];` crea dieci oggetti con il costruttore di default, liberati con `delete[] v;`.

6.3 Selettori, modificatori, `const` e `this`

Un **selettore** (*getter*) restituisce il valore di un attributo senza modificarlo; un **modificatore** (*setter*) lo cambia, controllando che il nuovo valore sia valido. I selettori si dichiarano **`const`** (la parola dopo i parametri): un metodo costante promette di non modificare l'oggetto, e il compilatore lo verifica. È necessario per chiamare metodi su oggetti costanti o passati per riferimento costante: su un `const Data& d` si può chiamare solo `d.stampa()` se `stampa` è `const`.

Dentro un metodo, **`this`** è un puntatore all'oggetto su cui il metodo è stato chiamato: `_g` equivale a `this->_g`. Serve per distinguere un attributo da un parametro con lo stesso nome (`this->x = x;`), per confrontare l'oggetto con un altro (`if (this != &v)`) e per restituire l'oggetto stesso (`return *this;`).

6.4 Il sovraccarico degli operatori

Per una classe si possono ridefinire gli operatori: `a + b` con `a` oggetto diventa la chiamata `a.operator+(b)`. L'operatore si dichiara come metodo (il primo operando è l'oggetto) oppure come funzione esterna con due parametri; se la funzione esterna deve accedere agli attributi privati la si dichiara **`friend`** della classe. Gli operatori `<<` e `>>` per i flussi devono essere funzioni esterne, perché il primo operando è `cout` (un `ostream`), e restituiscono il flusso per permettere le catene `cout << a << b`.

```cpp
// file: frazione.cpp
#include <iostream>
#include <cstdlib>
using namespace std;

class Frazione {
    int _num, _den;                    // _den > 0, ridotta ai minimi termini
    void semplifica();
public:
    Frazione(int num = 0, int den = 1); // fa anche da costruttore di default
    int getNum() const { return _num; }
    int getDen() const { return _den; }
    Frazione operator+(const Frazione& f) const;
    Frazione operator*(const Frazione& f) const;
    bool operator==(const Frazione& f) const;
    bool operator<(const Frazione& f) const;
    friend ostream& operator<<(ostream& os, const Frazione& f);
};

static int mcd(int a, int b) { return b == 0 ? a : mcd(b, a % b); }

void Frazione::semplifica() {
    if (_den < 0) { _num = -_num; _den = -_den; }
    int m = mcd(abs(_num), _den);
    if (m > 1) { _num /= m; _den /= m; }
}

Frazione::Frazione(int num, int den) : _num(num), _den(den == 0 ? 1 : den) {
    semplifica();
}
```

```cpp
Frazione Frazione::operator+(const Frazione& f) const {
    return Frazione(_num * f._den + f._num * _den, _den * f._den);
}

Frazione Frazione::operator*(const Frazione& f) const {
    return Frazione(_num * f._num, _den * f._den);
}

bool Frazione::operator==(const Frazione& f) const {
    return _num == f._num && _den == f._den;    // sono ridotte
}

bool Frazione::operator<(const Frazione& f) const {
    return _num * f._den < f._num * _den;       // denominatori positivi
}

ostream& operator<<(ostream& os, const Frazione& f) {
    os << f._num;
    if (f._den != 1) os << "/" << f._den;
    return os;
}

template <typename T>
T massimo(const T a[], int n) {
    T m = a[0];
    for (int i = 1; i < n; i++)
        if (m < a[i]) m = a[i];
    return m;
}

int main() {
    Frazione a(1, 6), b(-3, -4), c(4, -8);
    cout << a << " " << b << " " << c << endl;
    cout << a + b << " " << a * b << " " << (a + a + a == Frazione(1, 2))
         << " " << b + c + 2 << endl;
    Frazione v[] = {a, b, c, Frazione(5, 7)};
    cout << massimo(v, 4) << endl;
    return 0;
}
```

Alcuni dettagli. Gli argomenti di default (`int num = 0, int den = 1`) vanno nel prototipo e non nella definizione. In `b + c + 2` il 2 viene convertito in `Frazione(2)` dal costruttore con un parametro. Un metodo può accedere agli attributi privati di **un altro oggetto della stessa classe** (`f._den`): il controllo di accesso è per classe, non per oggetto. Il template `massimo` del modulo 5 funziona con le frazioni perché la classe definisce `<`.

6.5 Classi con memoria dinamica: la regola dei tre

Se una classe ha un attributo puntatore a memoria allocata con `new`, il costruttore di copia e l'assegnamento generati dal compilatore copiano il **puntatore**, non la memoria puntata (**copia superficiale**): due oggetti condividono lo stesso array, la modifica di uno si vede nell'altro, e quando entrambi vengono distrutti l'array viene liberato due volte. Serve la **copia profonda**, e per averla bisogna scrivere tre metodi insieme (**regola dei tre**):

1. il **distruttore**, che libera la memoria;
2. il **costruttore di copia**, che alloca un nuovo array e copia gli elementi;
3. l'**operatore di assegnamento**, che libera la memoria vecchia, alloca la nuova e copia, controllando l'**autoassegnamento** `a = a` (senza il controllo, liberando la memoria vecchia si perderebbero i dati da copiare).

L'assegnamento restituisce `*this`, così si possono scrivere catene `a = b = c`; le slide lo dichiarano con tipo di ritorno `const Vettore&`.

6.6 I template di classe

Come le funzioni, anche le classi possono avere un parametro di tipo. Il vettore dinamico è il caso tipico: lo stesso codice per vettori di interi, di reali, di frazioni.

```cpp
// file: vettore.cpp
#include <iostream>
using namespace std;

template <typename T>
class Vettore {
    int _dim;
    T* _val;
public:
    Vettore();
    Vettore(int dim, const T* pval);
    Vettore(const Vettore<T>& v);                    // copia profonda
    ~Vettore();
    int getDim() const { return _dim; }
    T& operator[](int i) { return _val[i]; }         // lettura e scrittura
    const T& operator[](int i) const { return _val[i]; }
    const Vettore<T>& operator=(const Vettore<T>& v);
    Vettore<T> operator+(const Vettore<T>& v) const; // elemento per elemento
};

template <typename T>
Vettore<T>::Vettore() : _dim(0), _val(nullptr) {}

template <typename T>
Vettore<T>::Vettore(int dim, const T* pval) : _dim(dim), _val(new T[dim]) {
    for (int i = 0; i < _dim; i++) _val[i] = pval[i];
}

template <typename T>
Vettore<T>::Vettore(const Vettore<T>& v) : _dim(v._dim), _val(new T[v._dim]) {
    for (int i = 0; i < _dim; i++) _val[i] = v._val[i];
}

template <typename T>
Vettore<T>::~Vettore() { delete[] _val; }

template <typename T>
const Vettore<T>& Vettore<T>::operator=(const Vettore<T>& v) {
    if (this != &v) {                    // a = a non deve fare nulla
        delete[] _val;
        _dim = v._dim;
        _val = new T[_dim];
        for (int i = 0; i < _dim; i++) _val[i] = v._val[i];
    }
    return *this;
}

template <typename T>
Vettore<T> Vettore<T>::operator+(const Vettore<T>& v) const {
    int n = _dim < v._dim ? _dim : v._dim;  // la parte comune
```

```
        Vettore<T> r(n, _val);
        for (int i = 0; i < n; i++) r._val[i] = r._val[i] + v._val[i];
        return r;
    }

    template <typename T>
    void stampa(const Vettore<T>& v) {
        for (int i = 0; i < v.getDim(); i++) cout << v[i] << " ";
        cout << endl;
    }

    int main() {
        int a[] = {1, 2, 3}, b[] = {10, 20, 30, 40};
        Vettore<int> x(3, a), y(4, b);
        Vettore<int> z(x);                  // costruttore di copia
        z[0] = 100;                         // x non cambia
        Vettore<int> w;
        w = x + y;
        w = w;                              // autoassegnamento
        stampa(x); stampa(z); stampa(w);
        double d[] = {0.5, 1.5};
        Vettore<double> u(2, d);
        stampa(u + u);
        return 0;
    }
```

Ogni metodo definito fuori dalla classe ricomincia con `template <typename T>` e usa il nome completo `Vettore<T>::`. Un template di classe si mette tutto in un file di intestazione, perché il compilatore deve vedere il codice per istanziarlo. Si usano due versioni di `operator[]`: quella non costante restituisce un riferimento e permette di scrivere `z[0] = 100`, quella costante serve sugli oggetti costanti, come il parametro di `stampa`.

Esempi ed esercizi svolti

> **Aggiunta in revisione · fonte: esercizi nuovi sul modello delle prove 2016-2020 (esercizi “identificare e correggere gli errori” e prova al calcolatore con una classe); programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-09)**

Esercizio 1. Identificare e correggere gli errori nella dichiarazione del template di classe.

```
1   template <typename T>
2   class Coppia
3   public:
4       Coppia(T a, T b);
5       Coppia(Coppia<T> c);
6       ~Coppia(int k);
7       T getPrimo();
8       void setPrimo(const T& a) const;
9       const Coppia<T>& operator=(const Coppia<T>& c);
10      bool operator==(const Coppia<T>& c) const;
11  private:
12      T _primo, _secondo
13  }
```

Riga 2: manca la graffa aperta, `class Coppia {`. Riga 5: il costruttore di copia riceve un riferimento costante, `Coppia(const Coppia<T>& c);`. Riga 6: il distruttore non ha parametri, `~Coppia();`. Riga 7: un selettore va dichiarato costante, `T getPrimo() const;`. Riga 8: un modificatore non può essere `const`, perché modifica l’oggetto. Riga 12: manca il `;`. Riga 13: dopo la graffa che chiude la classe serve `;`.

Esercizio 2. Quali metodi vengono chiamati? Per ogni riga, dire quale costruttore o operatore della classe `Vettore<int>` viene chiamato.

Codice	Chiamata
`Vettore<int> a;`	costruttore di default
`Vettore<int> b(3, v);`	costruttore con parametri
`Vettore<int> c = b;`	costruttore di copia (non l'assegnamento: `c` nasce ora)
`a = b;`	operatore di assegnamento
`stampa(b);` con parametro `const Vettore<int>&`	nessuno: si passa un riferimento
`void f(Vettore<int> x); f(b);`	costruttore di copia per il parametro, distruttore all'uscita
`Vettore<int>* p = new Vettore<int>(b);`	costruttore di copia; poi `delete p;` chiama il distruttore

Esercizio 3. La classe `Presenze` (sul modello della prova al calcolatore). Un corso tiene il registro delle presenze di `NS` studenti a `NL` lezioni. Si scriva la classe `Presenze` con: un codice del corso `_codice` (intero), il numero `_fatte` di lezioni già svolte (intero) e una matrice `_P` di `NS` righe e `NL` colonne, con `_P[i][j]` uguale a 1 se lo studente `i` era presente alla lezione `j`, 0 altrimenti. Metodi: costruttore di default (tutto a zero), costruttore con parametri, costruttore di copia, selettori (quello di `_P` riceve gli indici e restituisce -1 se non validi), modificatori, operatore di assegnamento; `presenzeStudente(i)` (somma della riga, -1 se l'indice non è valido); `presentiLezione(j)` (somma della colonna, -1 se non valido); `piuAssiduo()` (indice dello studente con più presenze, il più piccolo in caso di parità); `percentuali(double p[])`, che mette in `p[i]` la percentuale di presenza dello studente `i` sulle lezioni svolte; `leggiDati(nomeFile)`, che legge dalla prima riga codice e lezioni svolte, poi le `NS` righe della matrice. Il programma alloca dinamicamente un oggetto, legge il file, stampa lo studente più assiduo e quanti studenti sono sotto il 70% di presenze, e libera l'oggetto. Qui `NS` $= 4$ e `NL` $= 6$ per tenere piccolo il file di prova.

```cpp
// file: presenze.cpp
#include <iostream>
#include <fstream>
using namespace std;

const int NS = 4, NL = 6;

class Presenze {
    int _codice, _fatte;
    int _P[NS][NL];
public:
    Presenze();
    Presenze(int codice, int fatte, const int P[][NL]);
    Presenze(const Presenze& p);
    int getCodice() const { return _codice; }
    int getFatte() const { return _fatte; }
    int getP(int i, int j) const;
    void setCodice(int codice) { _codice = codice; }
    void setFatte(int fatte) { _fatte = fatte; }
    void setP(const int P[][NL]);
    const Presenze& operator=(const Presenze& p);
    int presenzeStudente(int i) const;
    int presentiLezione(int j) const;
    int piuAssiduo() const;
    void percentuali(double p[]) const;
    void leggiDati(const char nomeFile[]);
};

Presenze::Presenze() : _codice(0), _fatte(0) {
    for (int i = 0; i < NS; i++)
        for (int j = 0; j < NL; j++) _P[i][j] = 0;
```

```
}

Presenze::Presenze(int codice, int fatte, const int P[][NL])
    : _codice(codice), _fatte(fatte) {
    setP(P);
}

Presenze::Presenze(const Presenze& p)
    : _codice(p._codice), _fatte(p._fatte) {
    setP(p._P);
}

int Presenze::getP(int i, int j) const {
    if (i < 0 || i >= NS || j < 0 || j >= NL) return -1;
    return _P[i][j];
}

void Presenze::setP(const int P[][NL]) {
    for (int i = 0; i < NS; i++)
        for (int j = 0; j < NL; j++) _P[i][j] = P[i][j];
}

const Presenze& Presenze::operator=(const Presenze& p) {
    if (this != &p) {
        _codice = p._codice;
        _fatte = p._fatte;
        setP(p._P);
    }
    return *this;
}

// segue: presenze.cpp
int Presenze::presenzeStudente(int i) const {
    if (i < 0 || i >= NS) return -1;
    int s = 0;
    for (int j = 0; j < NL; j++) s += _P[i][j];
    return s;
}

int Presenze::presentiLezione(int j) const {
    if (j < 0 || j >= NL) return -1;
    int s = 0;
    for (int i = 0; i < NS; i++) s += _P[i][j];
    return s;
}

int Presenze::piuAssiduo() const {
    int best = 0;
    for (int i = 1; i < NS; i++)              // > stretto: vince il primo
        if (presenzeStudente(i) > presenzeStudente(best)) best = i;
    return best;
}

void Presenze::percentuali(double p[]) const {
    for (int i = 0; i < NS; i++)
        p[i] = _fatte > 0 ? 100.0 * presenzeStudente(i) / _fatte : 0;
}
```

```
void Presenze::leggiDati(const char nomeFile[]) {
    ifstream in(nomeFile);
    in >> _codice >> _fatte;
    for (int i = 0; i < NS; i++)
        for (int j = 0; j < NL; j++) in >> _P[i][j];
}

int main() {
    Presenze* pr = new Presenze;
    pr->leggiDati("presenze.txt");
    cout << "corso " << pr->getCodice() << ", piu' assiduo: "
         << pr->piuAssiduo() << endl;
    double perc[NS];
    pr->percentuali(perc);
    int sotto = 0;
    for (int i = 0; i < NS; i++)
        if (perc[i] < 70) sotto++;
    cout << sotto << " sotto il 70%, presenti alla lezione 2: "
         << pr->presentiLezione(2) << ", getP(9, 0) = " << pr->getP(9, 0)
         << endl;
    Presenze copia(*pr);                    // prova del costruttore di copia
    delete pr;
    cout << copia.presenzeStudente(3) << endl;
    return 0;
}
```

Con il file

```
104 5
1 1 0 1 1 0
1 1 1 1 1 0
0 1 0 0 1 0
1 1 1 1 0 0
```

stampa `corso 104, piu' assiduo: 1`, poi `1 sotto il 70%, presenti alla lezione 2: 2, getP(9, 0) = -1`, e infine 4. La copia resta valida dopo il `delete`, perché la matrice è un array dentro l'oggetto e il costruttore di copia l'ha duplicata.

Esercizio 4. Un vettore di frazioni. Si può usare `Vettore<Frazione>`? Che cosa serve alla classe `Frazione`?

Sì: `Vettore<T>` usa `new T[dim]`, che richiede il costruttore di default (c'è, grazie agli argomenti di default), l'assegnamento `=` (quello generato dal compilatore va bene, perché `Frazione` non ha memoria dinamica) e, per `operator+`, la somma di due `T` (c'è). Per stamparlo con `stampa` serve `<<` (c'è). È il senso dei template: il codice si adatta a ogni tipo che offre le operazioni usate.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-09)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. Rifare i conti di `Presenze`. Con il file dell'esercizio 3, calcolare a mano le uscite.

Passo 1: le somme delle righe. Studente 0: 4; studente 1: 5; studente 2: 2; studente 3: 4.

Passo 2: il più assiduo. Lo studente 1, con 5.

Passo 3: le percentuali su 5 lezioni svolte. 80%, 100%, 40%, 80%: sotto il 70% c'è un solo studente, il 2.

Passo 4: la colonna 2. $0 + 1 + 0 + 1 = 2$. `getP(9, 0)` ha un indice non valido: -1. Ultima riga: `copia.presenzeStudente(3)` vale 4.

Passo 5: la morale. Prima di consegnare un programma d'esame, si prepara un file piccolo di cui si conoscono a mano i risultati e si confronta.

Guidato 2. La copia superficiale. Che cosa succede se nella classe `Vettore` si toglie il costruttore di copia scritto a mano?

Passo 1: quello generato. Copia `_dim` e il **puntatore** `_val`.

Passo 2: l'effetto. In `Vettore<int> z(x); z[0] = 100;` anche `x[0]` diventerebbe 100, perché `x` e `z` condividono l'array.

Passo 3: la fine. All'uscita da `main` vengono distrutti sia `z` sia `x`, e lo stesso array viene liberato due volte: errore di esecuzione (AddressSanitizer lo segnala come *double free*). Per questo distruttore, costruttore di copia e assegnamento vanno scritti insieme.

Guidato 3. Un contatore di oggetti. Una classe `Contatore` tiene in un attributo `static` il numero di oggetti esistenti. Che cosa stampa il programma?

```cpp
// file: contatore.cpp
#include <iostream>
using namespace std;

class Contatore {
    static int _vivi;                // uno solo per tutta la classe
public:
    Contatore() { _vivi++; }
    Contatore(const Contatore&) { _vivi++; }
    ~Contatore() { _vivi--; }
    static int vivi() { return _vivi; }
};

int Contatore::_vivi = 0;            // definizione dell'attributo statico

void f(Contatore) { cout << Contatore::vivi() << " "; }   // per valore

int main() {
    Contatore a;
    {
        Contatore b, c;
        cout << Contatore::vivi() << " ";
    }                                // b e c distrutti qui
    f(a);                            // copia per il parametro
    Contatore* p = new Contatore[3];
    cout << Contatore::vivi() << " ";
    delete[] p;
    cout << Contatore::vivi() << endl;
    return 0;
}
```

Passo 1: il blocco interno. `a`, `b`, `c`: stampa 3; all'uscita dal blocco restano solo `a`.

Passo 2: la chiamata per valore. Il parametro (senza nome, perché `f` non lo usa) è una copia: dentro `f` gli oggetti sono 2; all'uscita la copia viene distrutta.

Passo 3: l'array dinamico. Tre oggetti in più con il costruttore di default: 4. Dopo `delete[]`, di nuovo 1.

Passo 4: l'uscita. `3 2 4 1`. Un parametro che non si usa si può lasciare senza nome, come nel costruttore di copia: il compilatore non avvisa e la copia viene fatta lo stesso.

Riepilogo

Una classe riunisce attributi e metodi; il controllo di accesso (`private`, `public`) realizza l'incapsulamento, e i metodi si definiscono fuori dalla classe con `Classe::`. I costruttori inizializzano gli oggetti (di default, con parametri, di copia con parametro `const T&`), preferibilmente con la lista di inizializzazione; il distruttore li chiude ed è chiamato all'uscita dal blocco o con `delete`. I selettori sono metodi `const`, i modificatori controllano i valori; `this` punta all'oggetto corrente. Gli operatori si ridefiniscono come metodi o come funzioni esterne (`friend` se servono gli attributi privati), e `<<` restituisce il flusso. Una classe con memoria dinamica deve avere distruttore, costruttore di copia e assegnamento con copia profonda e controllo dell'autoassegnamento. I template di classe parametrizzano il tipo degli elementi; ogni metodo esterno ripete `template <typename T>` e `Classe<T>::`.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (FI-09)**

Lo scheletro di una classe d'esame

```
class C {
    int _a;  T* _p;  int _m[R][K];          // attributi privati
public:
    C();                                     // default
    C(int a, ...);                           // con parametri
    C(const C& c);                           // di copia
    ~C();                                    // distruttore (se c'e' new)
    int getA() const;   void setA(int a);    // selettori, modificatori
    const C& operator=(const C& c);          // assegnamento
    friend ostream& operator<<(ostream& os, const C& c);
};
const C& C::operator=(const C& c) {
    if (this != &c) { /* libera, alloca, copia */ }
    return *this;
}
template <typename T> T Classe<T>::metodo(...) const { ... }
```

Situazione	Chi viene chiamato
`C x;` / `C x(…);`	costruttore di default / con parametri
`C y = x;`, `C y(x);`, passaggio o ritorno per valore	costruttore di copia
`y = x;` fra oggetti già esistenti	operatore di assegnamento
fine del blocco, `delete p`	distruttore
`new C[n]`	n volte il costruttore di default

Termini per l'orale

Termine	In una riga
Incapsulamento	attributi privati, accesso solo con i metodi
Interfaccia, implementazione	che cosa fa la classe; come lo fa
Costruttore di copia	crea un oggetto copia di un altro
Lista di inizializzazione	inizializza gli attributi prima del corpo
Metodo `const`	non modifica l'oggetto
`this`	puntatore all'oggetto corrente
`friend`	funzione esterna con accesso ai membri privati
Copia superficiale, profonda	copia del puntatore; della memoria puntata

Termine	In una riga
Regola dei tre	distruttore, copia e assegnamento si scrivono insieme
Membro `static`	appartiene alla classe, non al singolo oggetto

Domande che tornano spesso

- Differenza fra struttura e classe; che cos'è l'incapsulamento?
- Tipi di costruttore; quando viene chiamato il costruttore di copia e quando l'assegnamento?
- Perché il costruttore di copia riceve un riferimento? Perché l'assegnamento controlla `this != &c`?
- Come si ridefinisce <<, e perché non può essere un metodo?
- Che cos'è un template di classe e come si definiscono i suoi metodi fuori dalla classe?

7 · Tipi di dato astratti, liste, pile e code

In questo modulo. Si usano le classi, i template e la regola dei tre del modulo 6, e i puntatori e la memoria dinamica del modulo 5. È la parte di programmazione modulare con cui si chiude il corso. Si comincia dall'idea di **tipo di dato astratto**: un tipo definito da ciò che si può fare con i suoi valori, non da come è fatto dentro. Poi si studiano tre strutture dati fondamentali come tipi astratti: la **lista concatenata**, con inserimenti, rimozioni e ricerca; la **pila**, in cui l'ultimo elemento entrato è il primo a uscire; la **coda**, in cui esce per primo il primo entrato. Di pile e code si vedono due realizzazioni, con un array e con una lista, e un paio di applicazioni classiche. Alla fine del modulo si sanno scrivere e usare queste classi, disegnare che cosa succede ai puntatori durante un inserimento o una rimozione, e scegliere la realizzazione adatta.

> **Aggiunta in revisione · fonte: slide del corso (Fondamenti, secondo semestre: 4. Programmazione modulare e tipi di dato astratti, liste semplici, pile e code), `Esercitazioni++_stutture_dinamiche`, esercitazioni del 22 e 29 aprile (FI-10, FI-11)**
>
> Il modulo è scritto in revisione dal materiale del corso, con le convenzioni delle slide (classe `Node` con `_data` e `_next`, lista con `_head`). I programmi sono compilati ed eseguiti da `strumenti/verifica.py`, con AddressSanitizer per controllare che tutta la memoria venga liberata.

7.1 L'astrazione sui dati

Usando un `int` nessuno pensa al complemento a 2 e ai 32 bit: si usano la somma, il confronto, la stampa. Un **tipo di dato astratto** (ADT, *abstract data type*) generalizza questa idea: è un insieme di valori con un insieme di **operazioni**, descritto solo da che cosa fanno le operazioni. L'ADT "pila", per esempio, è definito da "inserisci in cima", "togli dalla cima", "guarda la cima", "è vuota?", e da una regola: si toglie sempre l'ultimo elemento inserito.

In C++ un ADT si realizza con una classe:

- l'**interfaccia** (la parte pubblica: prototipi dei metodi e loro descrizione) è ciò che conosce chi usa il tipo;
- la **rappresentazione** (gli attributi privati) e l'implementazione dei metodi sono nascoste: è l'**information hiding**.

Il guadagno è l'**indipendenza dalla rappresentazione**: si può cambiare il modo in cui i dati sono memorizzati (un array invece di una lista, un array ordinato invece di uno disordinato) senza cambiare una riga dei programmi che usano la classe, purché l'interfaccia resti la stessa. In 7.4 e 7.5 la stessa interfaccia di pila e di coda viene realizzata in due modi.

7.2 Le liste concatenate semplici

Un array ha dimensione fissa e, per inserire un elemento in mezzo, bisogna spostare tutti quelli che seguono. Una **lista concatenata** è una sequenza di **nodi** allocati dinamicamente, ciascuno con un dato e un puntatore al nodo **successivo**; l'ultimo nodo ha il puntatore nullo. La lista è individuata dal **puntatore di testa**, che punta al primo nodo (ed è nullo se la lista è vuota). Si può allungare e accorciare a piacere, e inserire o togliere un nodo è questione di spostare due puntatori; il prezzo è che per arrivare all'elemento i-esimo bisogna attraversare i precedenti.

```
_head -> [ 3 | * ] -> [ 8 | * ] -> [ 5 | 0 ]
```

Come nelle slide, la lista si realizza con due classi: `Node`, il singolo nodo, e `List`, l'intera lista.

```cpp
// file: lista.cpp
#include <iostream>
using namespace std;

class Node {
    int _data;
```

```
    Node* _next;
public:
    Node(int data = 0, Node* next = nullptr) : _data(data), _next(next) {}
    int getData() const { return _data; }
    Node* getNext() const { return _next; }
    void setData(int data) { _data = data; }
    void setNext(Node* next) { _next = next; }
};

class List {
    Node* _head;
    void copia(const List& l);            // aiuto per copia e assegnamento
    void svuota();                        // aiuto per distruttore e =
public:
    List() : _head(nullptr) {}
    List(const List& l);
    ~List();
    const List& operator=(const List& l);
    bool empty() const { return _head == nullptr; }
    int size() const;
    void insertFront(int x);
    void insertBack(int x);
    void insertAfter(Node* p, int x);
    void insertOrdered(int x);            // in una lista ordinata
    bool removeFront();
    bool remove(int x);                   // la prima occorrenza
    Node* search(int x) const;
    void print() const;
};
```

Inserimento in testa. Si crea un nodo che punta alla vecchia testa, e la testa diventa il nuovo nodo; funziona anche con la lista vuota. Costo costante.

Inserimento in coda. Se la lista è vuota è un inserimento in testa; altrimenti si scorre fino all'ultimo nodo (quello con `getNext()` nullo) e lo si fa puntare al nuovo nodo. Costo proporzionale alla lunghezza.

Inserimento dopo un nodo `p`. Il nuovo nodo punta al successivo di `p`, e `p` punta al nuovo nodo, **in quest'ordine**: facendo prima `p->setNext(nuovo)` si perderebbe il resto della lista.

```
// segue: lista.cpp
void List::insertFront(int x) { _head = new Node(x, _head); }

void List::insertBack(int x) {
    if (empty()) { insertFront(x); return; }
    Node* p = _head;
    while (p->getNext() != nullptr) p = p->getNext();
    p->setNext(new Node(x));
}

void List::insertAfter(Node* p, int x) {
    if (p == nullptr) return;
    p->setNext(new Node(x, p->getNext()));
}

void List::insertOrdered(int x) {
    if (empty() || x <= _head->getData()) { insertFront(x); return; }
    Node* p = _head;                      // p: ultimo nodo con dato < x
    while (p->getNext() != nullptr && p->getNext()->getData() < x)
        p = p->getNext();
```

```
    insertAfter(p, x);
}

bool List::removeFront() {
    if (empty()) return false;
    Node* q = _head;
    _head = _head->getNext();
    delete q;                           // prima si sposta la testa
    return true;
}

bool List::remove(int x) {
    if (empty()) return false;
    if (_head->getData() == x) return removeFront();
    Node* q = _head;                    // q: il nodo PRECEDENTE
    while (q->getNext() != nullptr && q->getNext()->getData() != x)
        q = q->getNext();
    if (q->getNext() == nullptr) return false;   // non c'e'
    Node* r = q->getNext();
    q->setNext(r->getNext());           // si "salta" r
    delete r;
    return true;
}
```

Rimozione. Per togliere un nodo bisogna modificare il puntatore del nodo **precedente**: per questo la ricerca si ferma un nodo prima, guardando `q->getNext()`. Il caso della testa è a parte, perché non ha un precedente e va modificato `_head`. In ogni caso si salva l'indirizzo del nodo da togliere, si sistemano i puntatori e solo dopo si fa `delete`.

```
// segue: lista.cpp
Node* List::search(int x) const {
    for (Node* p = _head; p != nullptr; p = p->getNext())
        if (p->getData() == x) return p;
    return nullptr;
}

int List::size() const {
    int n = 0;
    for (Node* p = _head; p != nullptr; p = p->getNext()) n++;
    return n;
}

void List::print() const {
    for (Node* p = _head; p != nullptr; p = p->getNext())
        cout << p->getData() << " ";
    cout << endl;
}

void List::svuota() { while (removeFront()) {} }

void List::copia(const List& l) {       // copia profonda, stesso ordine
    _head = nullptr;
    Node* ultimo = nullptr;
    for (Node* p = l._head; p != nullptr; p = p->getNext()) {
        Node* n = new Node(p->getData());
        if (ultimo == nullptr) _head = n;
        else ultimo->setNext(n);
        ultimo = n;
```

```
    }
}

List::List(const List& l) { copia(l); }
List::~List() { svuota(); }

const List& List::operator=(const List& l) {
    if (this != &l) { svuota(); copia(l); }
    return *this;
}

int main() {
    List l;
    l.insertBack(8); l.insertFront(3); l.insertBack(5);
    l.insertAfter(l.search(8), 6);
    l.print();                                  // 3 8 6 5
    List m(l);
    l.remove(8); l.remove(5); l.remove(42);
    l.print(); m.print();                       // m non cambia
    List o;
    int v[] = {7, 2, 9, 2, 4};
    for (int x : v) o.insertOrdered(x);
    o.print();
    m = o;
    cout << m.size() << " " << (m.search(9) != nullptr) << endl;
    return 0;
}
```

Il distruttore deve liberare **tutti** i nodi: con `delete _head` se ne libererebbe solo uno, e gli altri resterebbero allocati e irraggiungibili. Per la stessa ragione della classe `Vettore`, una lista ha bisogno di costruttore di copia e assegnamento con copia profonda. Il ciclo `for (Node* p = _head; p != nullptr; p = p->getNext())` è lo schema di **scansione** di una lista, l'analogo di `for (i = 0; i < n; i++)` per gli array. In `main`, `for (int x : v)` è il ciclo "per ogni elemento" del C++11 su un array.

7.3 La lista come template

La lista delle slide ha anche una versione generica, con `template <typename T> class Node` e `template <typename T> class List`: il codice è lo stesso, con `T` al posto di `int` per il dato e `Node<T>*` per i puntatori. I tipi astratti dei paragrafi seguenti sono scritti direttamente come template.

7.4 La pila

Una **pila** (*stack*) è una sequenza in cui si inserisce e si toglie sempre dallo stesso estremo, la **cima**: l'ultimo elemento inserito è il primo a uscire (**LIFO**, *last in, first out*), come una pila di piatti. Le operazioni sono `push` (inserisci in cima), `pop` (togli dalla cima), `top` (guarda la cima), `empty`. Le pile sono ovunque in informatica: lo stack delle chiamate di funzione del modulo 5, l'annulla degli editor, il controllo delle parentesi, la valutazione delle espressioni.

Con un array. Un array di capacità fissata e un indice `_top` del primo posto libero: `push` scrive in `_v[_top]` e incrementa, `pop` decrementa. Tutte le operazioni hanno costo costante; il limite è la capacità.

```
// file: pila.cpp
#include <iostream>
using namespace std;

template <typename T>
class Pila {                        // realizzazione con un array dinamico
    T* _v;
    int _cap, _top;                 // _top = numero di elementi
```

```
public:
    Pila(int cap = 100) : _v(new T[cap]), _cap(cap), _top(0) {}
    Pila(const Pila<T>& p) : _v(new T[p._cap]), _cap(p._cap), _top(p._top) {
        for (int i = 0; i < _top; i++) _v[i] = p._v[i];
    }
    ~Pila() { delete[] _v; }
    const Pila<T>& operator=(const Pila<T>& p);
    bool empty() const { return _top == 0; }
    bool full() const { return _top == _cap; }
    bool push(const T& x) {
        if (full()) return false;
        _v[_top++] = x;
        return true;
    }
    bool pop() {
        if (empty()) return false;
        _top--;
        return true;
    }
    T top() const { return _v[_top - 1]; }   // solo se non vuota
};

template <typename T>
const Pila<T>& Pila<T>::operator=(const Pila<T>& p) {
    if (this != &p) {
        delete[] _v;
        _cap = p._cap; _top = p._top;
        _v = new T[_cap];
        for (int i = 0; i < _top; i++) _v[i] = p._v[i];
    }
    return *this;
}
```

Applicazione: le parentesi bilanciate. Si scorre la stringa: una parentesi aperta va sulla pila; una chiusa deve corrispondere a quella in cima, che si toglie. Alla fine la pila deve essere vuota.

```
// segue: pila.cpp
bool bilanciata(const char s[]) {
    Pila<char> p;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') p.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (p.empty()) return false;             // chiusa di troppo
            char a = p.top();
            p.pop();
            if ((c == ')' && a != '(') || (c == ']' && a != '[') ||
                (c == '}' && a != '{')) return false;
        }
    }
    return p.empty();                                // aperte rimaste?
}

int valutaPostfissa(const char s[]) {   // cifre singole e + - *
    Pila<int> p;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c >= '0' && c <= '9') p.push(c - '0');
```

```
            else if (c == '+' || c == '-' || c == '*') {
                int b = p.top(); p.pop();              // l'ordine conta
                int a = p.top(); p.pop();
                p.push(c == '+' ? a + b : c == '-' ? a - b : a * b);
            }
        }
        return p.top();
    }

    int main() {
        cout << bilanciata("v[(i+1)*2] = {f(x)};") << " "
             << bilanciata("(a[0])") << " " << bilanciata("((x)") << endl;
        cout << valutaPostfissa("3 4 + 2 *") << " "
             << valutaPostfissa("9 5 3 - -") << endl;
        Pila<int> a(3), b;
        a.push(1); a.push(2); a.push(3);
        cout << a.push(4) << " ";                      // piena
        b = a;
        b.pop();
        cout << a.top() << " " << b.top() << endl;
        return 0;
    }
```

Nella **notazione postfissa** l'operatore segue gli operandi: `3 4 + 2 *` è $(3+4)\cdot 2 = 14$, e non servono parentesi. La pila la valuta da sinistra a destra: i numeri si impilano, un operatore toglie due operandi e impila il risultato. Attenzione all'ordine: il primo tolto è il secondo operando, e `9 5 3 - -` vale $9-(5-3)=7$.

Con una lista. La cima è la testa della lista: `push` è un inserimento in testa, `pop` una rimozione in testa, entrambi a costo costante, e la pila non ha capacità massima (esercizio 2).

7.5 La coda

Una **coda** (*queue*) è una sequenza in cui si inserisce da un estremo (il **fondo**) e si toglie dall'altro (la **testa**): esce per primo il primo entrato (**FIFO**, *first in, first out*), come la fila a uno sportello. Le operazioni sono `enqueue` (accoda), `dequeue` (togli dalla testa), `front` (guarda la testa), `empty`. Si usano per tutto ciò che va servito in ordine di arrivo: le stampe in attesa, i processi pronti di un sistema operativo, i pacchetti di rete.

Con una lista. Per avere entrambe le operazioni a costo costante si tengono **due puntatori**, alla testa e all'ultimo nodo: si accoda dopo l'ultimo, si toglie dalla testa. Bisogna curare i casi limite: accodando in una coda vuota il nuovo nodo è sia testa sia fondo; togliendo l'ultimo elemento anche il puntatore al fondo torna nullo.

```
// file: coda.cpp
#include <iostream>
using namespace std;

template <typename T>
class Coda {                            // realizzazione con una lista
    struct Nodo { T dato; Nodo* next; };
    Nodo *_head, *_tail;
    int _n;
public:
    Coda() : _head(nullptr), _tail(nullptr), _n(0) {}
    Coda(const Coda<T>& c) : _head(nullptr), _tail(nullptr), _n(0) {
        for (Nodo* p = c._head; p != nullptr; p = p->next) enqueue(p->dato);
    }
    ~Coda() { while (dequeue()) {} }
    const Coda<T>& operator=(const Coda<T>& c) {
        if (this != &c) {
            while (dequeue()) {}
```

```
            for (Nodo* p = c._head; p != nullptr; p = p->next)
                enqueue(p->dato);
        }
        return *this;
    }
    bool empty() const { return _head == nullptr; }
    int size() const { return _n; }
    void enqueue(const T& x) {
        Nodo* nuovo = new Nodo{x, nullptr};
        if (empty()) _head = nuovo;
        else _tail->next = nuovo;
        _tail = nuovo;
        _n++;
    }
    bool dequeue() {
        if (empty()) return false;
        Nodo* q = _head;
        _head = _head->next;
        if (_head == nullptr) _tail = nullptr;   // era l'ultimo
        delete q;
        _n--;
        return true;
    }
    T front() const { return _head->dato; }      // solo se non vuota
};

int main() {
    Coda<int> c;                // giro "a turno": ogni lavoro ha 3 unita'
    int resto[] = {5, 2, 7};    // di lavoro residuo per i lavori 0, 1, 2
    for (int i = 0; i < 3; i++) c.enqueue(i);
    while (!c.empty()) {
        int j = c.front();
        c.dequeue();
        resto[j] -= 3;
        cout << j;
        if (resto[j] > 0) c.enqueue(j);          // non finito: in fondo
        else cout << "*";                        // finito
        cout << " ";
    }
    cout << endl;
    Coda<char> a, b;
    for (char x = 'a'; x <= 'e'; x++) a.enqueue(x);
    b = a;
    b.dequeue(); b.dequeue();
    cout << a.size() << a.front() << " " << b.size() << b.front() << endl;
    return 0;
}
```

Il programma simula la politica **round robin** di un sistema operativo: ogni lavoro riceve a turno 3 unità di tempo e, se non ha finito, torna in fondo alla coda. Stampa `0 1* 2 0* 2 2*` (l'asterisco segna un lavoro finito), poi `5a 3c`. Il nodo qui è una `struct` dentro la classe: è un dettaglio della rappresentazione, invisibile a chi usa la coda.

Con un array circolare. Un array di capacità N, l'indice della testa e il numero di elementi; il fondo è in posizione $(testa + n) \bmod N$. Quando un indice arriva in fondo all'array ricomincia da 0, così lo spazio liberato in testa viene riusato. Tutte le operazioni hanno costo costante (guidato 2).

Struttura	Inserimento	Rimozione	Accesso all'elemento i	Limite
array	in fondo costante, in mezzo proporzionale a n	idem	costante	capacità fissa
lista semplice	in testa costante, in coda proporzionale a n	in testa costante	proporzionale a i	memoria per i puntatori
pila (array o lista)	`push` costante	`pop` costante	solo la cima	capacità (con l'array)
coda (lista con due puntatori, array circolare)	`enqueue` costante	`dequeue` costante	solo la testa	capacità (con l'array)

Esempi ed esercizi svolti

> **Aggiunta in revisione · fonte: esercizi nuovi sul modello delle prove 2016-2020 e delle esercitazioni sulle strutture dinamiche; programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-10, FI-11)**

Esercizio 1. Completare le affermazioni (sul modello della prova di teoria, su tutto il corso).

Affermazione	Completamento
Il puntatore di testa di una lista semplice punta al … della lista	primo nodo
In una pila si estrae per primo l'elemento inserito …	per ultimo (LIFO)
Un … identifica un insieme di oggetti e un insieme di operazioni su di essi	tipo di dato astratto
Un oggetto allocato … va esplicitamente deallocato	dinamicamente (con `new`)
L'istruzione … termina la funzione in esecuzione	`return`
Dato un puntatore a un oggetto, si accede ai membri con l'operatore …	`->`
Il numero di relazioni fra gli insiemi finiti A e B vale …	$2^{\lvert A\rvert \lvert B\rvert}$
Nei cicli definiti il numero di ripetizioni è controllato da un …	contatore
Un elemento v confrontabile con ogni $x \in X$ e minore o uguale di ognuno è un … di X	minorante
In una coda realizzata con una lista, l'inserimento avviene in …	fondo (dopo l'ultimo nodo)

Esercizio 2. La pila con una lista. Realizzare l'interfaccia della pila del paragrafo 7.4 con nodi collegati, senza limite di capacità.

```cpp
// file: pila_lista.cpp
#include <iostream>
using namespace std;

template <typename T>
class PilaL {
    struct Nodo { T dato; Nodo* next; };
    Nodo* _cima;
public:
    PilaL() : _cima(nullptr) {}
    PilaL(const PilaL<T>& p) : _cima(nullptr) { copia(p._cima); }
    ~PilaL() { while (pop()) {} }
    const PilaL<T>& operator=(const PilaL<T>& p) {
```

```
        if (this != &p) { while (pop()) {} copia(p._cima); }
        return *this;
    }
    bool empty() const { return _cima == nullptr; }
    void push(const T& x) { _cima = new Nodo{x, _cima}; }   // in testa
    bool pop() {
        if (empty()) return false;
        Nodo* q = _cima;
        _cima = _cima->next;
        delete q;
        return true;
    }
    T top() const { return _cima->dato; }
private:
    void copia(Nodo* p) {          // ricorsiva: copia dal fondo alla cima
        if (p == nullptr) return;
        copia(p->next);
        push(p->dato);
    }
};

int main() {
    PilaL<int> p;
    for (int i = 1; i <= 5; i++) p.push(i * i);
    PilaL<int> q(p);
    p.pop(); p.pop();
    cout << p.top() << " " << q.top() << " ";
    while (!q.empty()) { cout << q.top() << " "; q.pop(); }
    cout << endl;
    return 0;
}
```

Stampa `9 25 25 16 9 4 1`: gli elementi escono in ordine inverso di inserimento. La copia è delicata: scorrendo la pila originale dalla cima e facendo `push`, la copia verrebbe rovesciata; la funzione ricorsiva `copia` arriva prima in fondo e inserisce risalendo. L'interfaccia è la stessa di `Pila` (salvo `full`, che qui non serve): chi usa la pila non si accorge del cambio di rappresentazione.

Esercizio 3. Metodi nuovi per la lista. Aggiungere alla lista di interi due funzioni: la somma degli elementi e il rovesciamento della lista sul posto, senza allocare nodi.

```
// file: lista_rovescia.cpp
#include <iostream>
using namespace std;

struct Nodo { int dato; Nodo* next; };

void inserisciTesta(Nodo*& testa, int x) { testa = new Nodo{x, testa}; }

int somma(const Nodo* p) {      // ricorsiva
    return p == nullptr ? 0 : p->dato + somma(p->next);
}

void rovescia(Nodo*& testa) {
    Nodo* prec = nullptr;
    Nodo* p = testa;
    while (p != nullptr) {
        Nodo* succ = p->next;     // si salva il resto
        p->next = prec;           // si gira la freccia
        prec = p;
```

```
            p = succ;
        }
        testa = prec;
    }

    void stampa(const Nodo* p) {
        for (; p != nullptr; p = p->next) cout << p->dato << " ";
        cout << endl;
    }

    void distruggi(Nodo*& testa) {
        while (testa != nullptr) {
            Nodo* q = testa;
            testa = testa->next;
            delete q;
        }
    }

    int main() {
        Nodo* l = nullptr;
        for (int i = 1; i <= 5; i++) inserisciTesta(l, 10 * i);
        stampa(l);
        rovescia(l);
        stampa(l);
        cout << somma(l) << endl;
        distruggi(l);
        return 0;
    }
```

Stampa `50 40 30 20 10`, poi `10 20 30 40 50` e `150`. Qui la lista è scritta “alla C”, con una `struct` e funzioni esterne, come in alcune esercitazioni: il puntatore di testa si passa **per riferimento** (`Nodo*&`) alle funzioni che lo devono cambiare. Il rovesciamento usa tre puntatori: se si girasse la freccia di `p` senza aver salvato `p->next`, il resto della lista andrebbe perso.

Esercizio 4. Disegnare i puntatori. Una lista contiene `3 8 6 5`; si esegue `remove(6)`. Descrivere i passi.

`q` parte dalla testa (3); `q->getNext()` è 8, diverso da 6: `q` avanza a 8. Ora `q->getNext()` è 6: ci si ferma. `r` = nodo 6; `q->setNext(r->getNext())` fa puntare 8 a 5; `delete r`. Lista: `3 8 5`. Se si facesse `delete r` prima di leggere `r->getNext()`, si leggerebbe memoria già liberata.

Esercizi guidati

> **Aggiunta in revisione · fonte: esercizi nuovi; programmi compilati ed eseguiti con `strumenti/verifica.py` (FI-10, FI-11)**
>
> Ogni esercizio è diviso in passi: si prova un passo alla volta e si confronta solo dopo.

Guidato 1. Pila e coda a confronto. Si inseriscono nell’ordine 1, 2, 3, 4 in una pila e in una coda, poi si tolgono due elementi, si inserisce 5 e si tolgono tutti. In che ordine escono?

Passo 1: la pila. Dopo gli inserimenti la cima è 4: escono 4 e 3; resta `1 2` (cima 2); entra 5; escono 5, 2, 1. Ordine: `4 3 5 2 1`.

Passo 2: la coda. Escono 1 e 2; resta `3 4`; 5 va in fondo: escono 3, 4, 5. Ordine: `1 2 3 4 5`.

Passo 3: la regola. La coda conserva l’ordine di arrivo, la pila lo rovescia: per rovesciare una sequenza basta farla passare da una pila.

Guidato 2. La coda con un array circolare.

Passo 1: la rappresentazione. Array `_v` di capacità `_cap`, indice `_testa` del primo elemento, numero di elementi `_n`.

Passo 2: le operazioni. `enqueue` scrive in `(_testa + _n) % _cap` e incrementa `_n`; `dequeue` porta `_testa` a `(_testa + 1) % _cap` e decrementa `_n`. Vuota se `_n == 0`, piena se `_n == _cap`.

```cpp
// file: coda_circolare.cpp
#include <iostream>
using namespace std;

const int CAP = 4;

class CodaC {
    int _v[CAP];
    int _testa, _n;
public:
    CodaC() : _testa(0), _n(0) {}
    bool empty() const { return _n == 0; }
    bool full() const { return _n == CAP; }
    bool enqueue(int x) {
        if (full()) return false;
        _v[(_testa + _n) % CAP] = x;
        _n++;
        return true;
    }
    bool dequeue() {
        if (empty()) return false;
        _testa = (_testa + 1) % CAP;
        _n--;
        return true;
    }
    int front() const { return _v[_testa]; }
    int posTesta() const { return _testa; }
};

int main() {
    CodaC c;
    for (int x = 1; x <= 5; x++) cout << c.enqueue(x);   // il 5 non entra
    cout << endl;
    c.dequeue(); c.dequeue();
    c.enqueue(6); c.enqueue(7);                          // girano in testa
    cout << c.posTesta() << ":";
    while (!c.empty()) { cout << " " << c.front(); c.dequeue(); }
    cout << endl;
    return 0;
}
```

Passo 3: la prova. La prima riga è `11110`: il quinto inserimento fallisce. Dopo due `dequeue` la testa è in posizione 2; 6 e 7 vanno nelle posizioni 0 e 1, liberate. Escono `3 4 6 7`. Senza il modulo, dopo quattro inserimenti la coda non potrebbe più accettare elementi anche con l'array mezzo vuoto.

Guidato 3. Un ADT con due rappresentazioni. L'ADT "insieme di interi fra 0 e 99" ha le operazioni `aggiungi`, `togli`, `contiene`, `cardinalita`. Confrontare due rappresentazioni.

Passo 1: array di booleani. `bool _c[100]`, con `_c[x]` vero se `x` è nell'insieme: `aggiungi`, `togli` e `contiene` costano un accesso; `cardinalita` richiede di contare (o un contatore in più).

Passo 2: array ordinato degli elementi. `int _e[100]` più il numero di elementi: `contiene` con la ricerca binaria del modulo 4 (circa $\log_2 n$ confronti), `aggiungi` e `togli` devono spostare gli elementi per mantenere l'ordine; occupa meno memoria se gli elementi sono pochi e l'intervallo è grande.

Passo 3: l'interfaccia. In entrambi i casi la parte pubblica della classe è la stessa (i quattro metodi con gli stessi prototipi): un programma che usa l'insieme non cambia se si passa da una rappresentazione all'altra. È l'indipendenza dalla rappresentazione, il motivo per cui gli attributi sono privati.

Riepilogo

Un tipo di dato astratto è definito dalle sue operazioni; in C++ è una classe con interfaccia pubblica e rappresentazione privata (information hiding), che si può cambiare senza toccare il codice che la usa. Una lista concatenata semplice è una catena di nodi con dato e puntatore al successivo, a partire dal puntatore di testa: inserimenti e rimozioni spostano puntatori, nell'ordine giusto, e la rimozione richiede il nodo precedente; la scansione segue i puntatori fino a quello nullo; il distruttore libera tutti i nodi, e servono copia e assegnamento profondi. La pila (LIFO) inserisce e toglie dalla cima, con un array o in testa a una lista, e serve per le chiamate, le parentesi e la notazione postfissa. La coda (FIFO) accoda in fondo e toglie dalla testa, con una lista con due puntatori o un array circolare, e serve per servire in ordine di arrivo.

Formulario

> **Aggiunta in revisione · fonte: sintesi del modulo (FI-10, FI-11)**

Schemi sulle liste

```
scansione:      for (Node* p = _head; p != nullptr; p = p->getNext())
in testa:       _head = new Node(x, _head);
dopo p:         p->setNext(new Node(x, p->getNext()));
togli la testa: Node* q = _head; _head = _head->getNext(); delete q;
togli dopo q:   Node* r = q->getNext(); q->setNext(r->getNext()); delete r;
coda, accoda:   if (vuota) _head = n; else _tail->next = n; _tail = n;
coda, togli:    ... if (_head == nullptr) _tail = nullptr;
circolare:      fondo = (testa + n) % CAP;   testa = (testa + 1) % CAP;
```

ADT	Regola	Operazioni
pila	LIFO	`push`, `pop`, `top`, `empty`
coda	FIFO	`enqueue`, `dequeue`, `front`, `empty`
lista	sequenza con accesso per scansione	inserimenti, rimozioni, ricerca, scansione

Termini per l'orale

Termine	In una riga
Tipo di dato astratto	valori più operazioni, indipendente dalla rappresentazione
Information hiding	rappresentazione nascosta dietro l'interfaccia
Nodo	dato più puntatore al successivo
Puntatore di testa	punta al primo nodo; nullo se la lista è vuota
LIFO, FIFO	l'ultimo entrato esce per primo; il primo entrato esce per primo
Array circolare	gli indici ripartono da 0 con il modulo
Notazione postfissa	operatore dopo gli operandi, valutata con una pila

Domande che tornano spesso

- Che cos'è un tipo di dato astratto; che cosa vuol dire indipendenza dalla rappresentazione?
- Differenze fra array e lista; costo delle operazioni.
- Come si inserisce e si toglie un nodo; perché l'ordine delle istruzioni conta?
- Pila e coda: definizione, operazioni, realizzazione con array e con lista, applicazioni.

- Perché una classe lista ha bisogno di distruttore, costruttore di copia e assegnamento?